National University of Computer and Emerging Sciences, Lahore

GeoSync

Zainab Rashid 20L-0965 BS(CS)

Hadeeqa Imran 20L-1004 BS(CS)

Huzaifa Bin Faheem 20L-1133 BS(CS)

Supervisor: Dr. Kashif Zafar

Final Year Project

June 10, 2024

Anti-Plagiarism Declaration

This is to declare that the above publication was produced under the:

Title: GeoSync

is the sole contribution of the author(s), and no part hereof has been reproduced as it is the basis (cut and paste) that can be considered Plagiarism. All referenced parts have been used to argue the idea and cited properly. I/We will be responsible and liable for any consequence if a violation of this declaration is determined.

Date: June 5, 2024

Name: Zainab Rashid

Signature:

Name: Hadeeqa Imran

Signature:

Name: Huzaifa Bin Faheem

Signature:

Author's Declaration

This states Authors' declaration that the work presented in the report is their own, and has not been submitted/presented previously to any other institution or organization.

Abstract

In today's fast-paced and ever-evolving world, the challenge of keeping location data up-to-date has become increasingly complex and critical, especially within the context of Careem services. The problem stems from the fact that the attributes of Points of Interests (POIs), specifically names and addresses, are not static; they change over time due to various reasons, including business rebranding, relocations, or closures. Consequently, relying on outdated or inaccurate location data can lead to various issues, from lost customers and business inefficiencies to frustrating user experiences in navigation and search applications. To address this challenge for Careem and ensure the reliability and relevance of location-based information in our digital age, we propose a hybrid approach to ensure a robust system for maintaining location data accuracy. By addressing this critical issue, we aim to enhance the overall user experience and optimize the accuracy of location-based services for Careem.

Executive Summary

In today's dynamic world, the challenge of maintaining accurate location data, especially within the context of Careem services, has become increasingly vital. Locations change over time due to various reasons, creating potential issues ranging from lost customers to operational inefficiencies. To address this challenge, we propose a hybrid approach that combines web crawlers, feedback from Careem Captains, and open-source maps to ensure the accuracy and relevance of location data.

The purpose of this document, as discussed in the first chapter, is to explore the automation of Points of Interest (POI) change detection on a large scale for ride-sharing services. The project's goals are outlined in the second chapter. These include developing an effective web crawling algorithm, exploring open source maps for acquiring upto-date location data, utilizing historical data patterns, and implementing a feedback system for captains. By achieving these objectives, the project aims to create a comprehensive solution that enhances the user experience and operational efficiency for Careem.

The document proceeds to delve into an extensive literature review in the third chapter, providing comprehensive insights into previous research conducted within our problem domain. Key components highlighted by the literary works, which could lead us to a potential solution, include acquiring high-quality location data and implementing change detection mechanisms, while also cultivating ways for future work in the domain of data validation. In addition, the literature review includes existing applications which use similar approaches to our proposed solution.

The fourth chapter discusses the functional and non-functional requirements, along with other software and hardware prerequisites necessary for the development of our system. Additionally, the chapter provides an in-depth analysis of use cases as well as the graphical user interface (GUI). Finally, potential risks associated with this project are also examined.

The proposed approach and methodology are outlined in the fifth chapter. The Geosync system involves three key modules- pattern recognition, web scraping, and a driver feedback system- dedicated to detecting changes in specified drop-off locations. This chapter provides a detailed overview of each module, including its functionality and employed algorithms.

The sixth chapter provides an overview of the GeoSync system, its high-level and low-level designs, and the interactions between various system components. These elements lay the groundwork for the development and implementation of GeoSync, aiming to deliver a robust and user-friendly system.

The seventh chapter marks a significant milestone in the project's progression toward a comprehensive solution. It details the full implementation of the GeoSync system, including its open-source data acquisition, and the development of the driver feedback mechanism, web crawler, and pattern detection

modules. The chapter illustrates the transformation of the theoretical framework into an operational system and provides test case tables assessing each module's accuracy, efficiency, and integration, covering metrics such as accuracy, efficiency, and integration. These test cases ensure the robustness and reliability of GeoSync, confirming that all modules work cohesively.

The eighth chapter provides a comprehensive user manual for the GeoSync system. This manual guides users through the setup and operation of the system, including detailed instructions on how to navigate the graphical user interface (GUI) and use each module effectively. This chapter aims to ensure that users can seamlessly adopt and utilize GeoSync, maximizing its value for Careem services.

To conclude, the project has successfully addressed the challenges of detecting changes in location data in ride-sharing services. All modules, including data acquisition, driver feedback, web crawling, and pattern detection, have been integrated and tested, ensuring that GeoSync is ready for deployment. This will enhance the efficiency and user experience for Careem by streamlining its operations and providing reliable location-based services.

Table of Contents

List of Figures **xi**

List of Tables **xii**

1 Introduction **1**

1.1 Purpose of this Document 1

1.2 Intended Audience 2

1.3 Definitions, Acronyms, and Abbreviations 2

1.3.1 Definitions 2

1.3.2 Acronyms and Abbreviations 2

1.4 Conclusion 2

2 Project Vision **4**

2.1 Problem Domain Overview 4

2.2 Problem Statement 4

2.3 Problem Elaboration 4

2.3.1 Manual Updation 4

2.3.2 Repetitive Task 5

2.3.3 Resource Allocation 5

2.3.4 Data Reliability and Timeliness 5

2.4 Goals and Objectives 5

2.5 Project Scope 6

2.6 Sustainable Development Goal (SDG) 6

2.6.1 Decent Work and Economic Growth 6

2.6.2 Industry, Innovation, and Infrastructure 6

3 Literature Review / Related Work **8**

3.1 Definitions, Acronyms and Abbreviations 8

3.1.1 Definitions 8

3.1.2 Acronyms and Abbreviations . . . 8
3.2 Detailed Literature Review . . . 9
3.2.1 Web Crawling . . . 9
3.2.2 Acquiring Open Source Location Data . . . 11
3.2.3 Pattern Detection . . . 13
3.2.4 Future Work . . . 16
3.2.5 Related Applications . . . 20

4 Software Requirement Specifications 24
4.1 List of Features . . . 24
4.2 Functional Requirements . . . 24
4.2.1 OSM Data Collection . . . 24
4.2.2 Location Data Repository Obtained Using Web Crawling . . . 24
4.2.3 Real-Time Use of Web Crawling . . . 24
4.2.4 Pattern Detection to Identify Location Changes . . . 25
4.2.5 Driver Feedback Mechanism . . . 25
4.2.6 Notification System . . . 25
4.3 Quality Attributes . . . 25
4.4 Non-Functional Requirements . . . 25
4.4.1 Performance . . . 25
4.4.2 Readability . . . 25
4.4.3 Availability . . . 26
4.4.4 Reusability of Change Detection Modules . . . 26
4.5 Assumptions . . . 26
4.6 Use Cases . . . 26
4.6.1 Sign Up . . . 26
4.6.2 Login . . . 27
4.6.3 Detect Location Change . . . 27
4.6.4 Notify User . . . 28
4.6.5 Provide Feedback . . . 28
4.7 Hardware and Software Requirements . . . 29
4.7.1 Hardware Requirements . . . 29
4.7.2 Software Requirements . . . 29
4.8 Graphical User Interface . . . 30
4.8.1 Signup page . . . 30

4.8.2 Login page 31
4.8.3 Home Page 32
4.8.4 Notification Page 32
4.8.5 Driver Feedback Page 33
4.9 Risk Analysis 33
4.9.1 Anti-Crawling Measures 33
4.9.2 Algorithmic Complexity 33
4.9.3 Integration Challenges 33
4.9.4 Dependency on External Platforms 34
4.9.5 Feedback System Reliability 34
4.10 Conclusion 34

5 Proposed Approach and Methodology 35
5.1 Acquisition of Open Source Location Data 35
5.2 Change Detection 36
5.2.1 Pattern Detection 36
5.2.2 Web Crawler 37
5.2.3 Driver Feedback Mechanism 37
5.3 Conclusion 38

6 High-Level and Low-Level Design 39
6.1 System Overview 39
6.1.1 Change Detection Module 39
6.2 Design Considerations 40
6.2.1 Assumptions and Dependencies 40
6.2.2 General Constraints 41
6.2.3 Goals and Guidelines 41
6.2.4 Development Methods 42
6.3 System Architecture 42
6.3.1 Pattern Detector 42
6.3.2 Web Crawler 44
6.3.3 Driver Feedback Module 45
6.4 Architectural Strategies 45
6.4.1 Tools and Software 45
6.4.2 Client - Server Model 46

6.4.3 Database 46
6.4.4 Interface Paradigms 46
6.4.5 Future Plans for Extension 46
6.5 Domain Model/Class Diagram 46
6.6 Sequence Diagrams 47
6.6.1 Geosync System 47
6.6.2 Pattern Detector 48
6.6.3 Web Crawler 48
6.6.4 Driver Feedback Mechanism 49
6.7 Policies and Tactics 49
6.7.1 Algorithm Selection for Pattern Detection 50
6.7.2 Testing Strategy 50
6.7.3 Coding Guidelines 50
6.7.4 Django as the Framework Choice 50
6.7.5 Plans for Ensuring Requirements Traceability 50
6.8 Conclusion 51

7 Implementation and Test Cases 52
7.1 Implementation 52
7.1.1 Dataset Aquisition 52
7.1.2 Driver Feedback 53
7.1.3 Pattern Detection 54
7.1.4 Web Crawler 55
7.2 Test Case Design and Description 56
7.2.1 Rider Login Test Case 56
7.2.2 Driver Login Test Case 57
7.2.3 Rider Signup Test Case 57
7.2.4 Driver Signup Test Case 58
7.2.5 Book Ride Test Case 59
7.2.6 Notify Rider Test Case 60
7.2.7 Driver Feedback Test Case 60
7.2.8 Pattern Detector Test Case 61
7.2.9 Web Crawler Test Case 62
7.2.10 Change Detector Test Case 63
7.2.11 Test Cases Summary Table 63

7.3 Conclusion 64

8 User Manual **65**

8.1 Introduction of Geosync 65

8.1.1 Overview of Change Detector System 65

8.1.2 Purpose and Benefits of Change Detector System 65

8.2 User Interaction 65

8.2.1 Registration and Account Creation 65

8.2.2 Login for Drivers and Riders 66

8.2.3 Navigating the Website 67

9 Conclusion **69**

List of Figures

2.1 **Decent Work Ethic Sustainable Development Goal** 7
2.2 **Industry Innovation and Infrastructure Sustainable Development Goal** 7

4.1 **Signup Page** 30
4.2 **Rider Signup Page** 30
4.3 **Login Page** 31
4.4 **User Login Page** 31
4.5 **Home Page** 32
4.6 **Notification Page** 32
4.7 **Driver Feedback Page** 33

5.1 **OSM Data Collection Strategy** 35
5.2 **Siamese GNN Module Design** 37

6.1 **High-level Architecture Diagram** 43
6.2 **Pattern Detector - Subsystem Architecture Diagram** 44
6.3 **Web Crawler - Subsystem Architecture Diagram** 44
6.4 **Driver Feedback - Subsystem Architecture Diagram** 45
6.5 **Class Diagram** 47
6.6 **GeoSync Design Sequence Diagram** 47
6.7 **Pattern Detector Design Sequence Diagram** 48
6.8 **Web Crawler Design Sequence Diagram** 48
6.9 **Driver Feedback Design Sequence Diagram** 49

8.1 **Signup Form** 66
8.2 **Signup Page** 66
8.3 **Login Page** 67
8.4 **Driver Homepage** 68
8.5 **Rider Homepage** 68

List of Tables

3.1 **Web Crawling Literature Review Summary Table** 11
3.2 **Acquiring Open Source Location Data Literature Review Summary Table** 13
3.3 **Pattern Detection Literature Review Summary Table** 16
3.4 **Future Work Literature Review Summary Table** 20
3.5 **Related Applications Summary Table** 23

Chapter 1 Introduction

In the contemporary, rapidly changing environment, the task of ensuring precise and current location information has grown progressively intricate and indispensable. Ride-sharing services, specifically Careem, require an algorithmic way to automate the process of keeping Points of Interest (POI) up-to-date. The heart of this challenge lies in the dynamic nature of location attributes, such as names and addresses, which are subject to constant change due to factors like business rebranding, relocations, and closures. This dynamic landscape presents a substantial problem, as relying on outdated or inaccurate location data can result in a myriad of issues, including lost customers, operational inefficiencies, and frustrating user experiences in navigation and search applications.

To address this pressing issue and ensure the reliability and relevance of location-based information in the digital age, we propose a hybrid solution that integrates multiple ways of detecting location changes. Our multifaceted approach includes harnessing the power of web crawlers to scour the internet for possible updates, utilizing open-source maps and historical data patterns to track location changes, and using location discrepancy feedback from Careem Captains.

By tackling this challenge head-on, we aim to improve the accuracy of location data for Careem, ultimately enhancing the overall user experience and operational efficiency of these ride-sharing services. This project represents a pivotal step towards ensuring that the information guiding these services remains as dynamic as the world they navigate.

The first section of the project outlines the purpose and target audience of the research. The subsequent chapter defines the project's scope and its main goals. In the third chapter, a comprehensive review of the existing literature is provided.

1.1 Purpose of this Document

The purpose of this document is to present the design, implementation, and evaluation of our project, which addresses the critical issue of detecting outdated Points of Interest (POI) data for ride-sharing services, specifically Careem. In today's rapidly evolving landscape, the need for precise location information is paramount, especially in the context of passenger requests and efficient navigation. Outdated or inaccurate location data can lead to customer dissatisfaction, operational inefficiencies, and navigation challenges. Our project aims to tackle this challenge by proposing a multifaceted solution that integrates web crawlers, feedback from Careem Captains, and open-source maps to keep location data current and reliable.

The research question we aim to explore is: Is there a robust way to automate POI change detection on a large scale? Through this endeavor, we aspire to enhance the overall user experience and operational

efficiency of ride-sharing services, contributing to a more dynamic and dependable digital environment for both businesses and users alike.

1.2 Intended Audience

The intended audience for this project is Careem and consequently the people using their services. Careem's interest lies in minimizing the resources they currently employ for detection and updation of change in location data. They also aim to enhance the customer experience and minimize inconvenience caused by discrepancies in the location data. This project presents a comprehensive solution to their problem, and aims at improving the quality of their service.

1.3 Definitions, Acronyms, and Abbreviations

Definitions of key terms along with expansions of abbreviations and acronyms are given below.

1.3.1 Definitions

Points of Interest: A location that has a special or particular interest for a particular purpose e.g. a tourist attraction or a recreational site

Careem captains: Drivers who provide transportation services through the platform

Web crawler: the program responsible for collecting web content by following hyperlinks

Open-source: A type of software or project that is made freely available to the public with its source code accessible and modifiable by anyone

Ride-sharing: A transportation service that connects passengers with drivers willing to provide one-way rides on short notice

1.3.2 Acronyms and Abbreviations

SDG: Sustainable Development Goal

FYP: Final Year Project

IEEE: Institute of Electrical and Electronics Engineers

POI: Point of Interest

1.4 Conclusion

To conclude, this chapter provides a foundational understanding of the critical challenge faced in maintaining accurate and detecting outdated POI data for Careem. The contemporary landscape demands

precise location information to ensure seamless navigation and enhanced user experiences. The dynamic nature of location attributes poses substantial issues if not managed effectively. Our proposed hybrid solution, integrating web crawlers, feedback from drivers (Careem Captains), and open-source maps, aims to revolutionize POI data maintenance. By addressing this challenge, we endeavor to significantly improve the reliability and efficiency of ride-sharing services, ultimately benefiting both Careem and its users. This chapter sets the stage for a comprehensive exploration of our multifaceted approach in subsequent sections, aligning with Careem's vision of a more dynamic and dependable digital environment.

Chapter 2 Project Vision

In this chapter, the project vision is unveiled, presenting a solution to the intricacies of keeping location data up-to-date in Careem's system. The primary objective is to automate the manual procedures presently in place.

2.1 Problem Domain Overview

Our hybrid system automates the labor-intensive and time-consuming process of keeping Careem's location data up-to-date. Different components such as web crawling, cross-referencing with open source maps, pattern detection and driver feedback are deployed simultaneously in order to detect changes in POIs. Through these components, the system ensures the accuracy, reliability, and timeliness of location data, resulting in an enhanced user experience and improved operational efficiency for both Careem and its users.

2.2 Problem Statement

The problem at hand involves the manual updation of locations by Careem employees, a repetitive and resource-intensive task. Each time a location changes, human resources must be allocated to monitor and update these changes in Careem's system. This inefficiency not only consumes valuable human resources but also risks outdated data. To address these issues, we propose automating this labor-intensive process, ultimately ensuring the reliability and timeliness of location data for Careem and their users.

2.3 Problem Elaboration

The problem at hand revolves around the manual process of updating location data within Careem. This process is characterized by several critical issues described below.

2.3.1 Manual Updation

The first challenge is the reliance on manual methods for updating location data. Careem employees are tasked with the responsibility of identifying and implementing changes in the location data within the system. This process is time-consuming and demands substantial human effort.

2.3.2 Repetitive Task

Location data updates are a recurring requirement. As locations change or new ones are added, the same process of manual verification, updating, and validation must be repeated. This repetition not only consumes significant human resources but also exposes the system to the risk of errors due to the monotony of the task.

2.3.3 Resource Allocation

The allocation of human resources for this manual process is a substantial cost for the organization. Valuable human capital is being utilized for tasks that could potentially be automated, diverting these resources away from more strategic and value-added activities.

2.3.4 Data Reliability and Timeliness

The manual nature of this process also introduces the risk of outdated and erroneous location data. As changes occur in the real world, there may be delays in updating the data in Careem's systems, leading to discrepancies between the digital representation and reality. Such inaccuracies can negatively impact user experiences and operational efficiency.

2.4 Goals and Objectives

The goals and objectives of this project, which addresses the challenge of detecting outdated location data for Careem, are as follows:

1. Develop and implement an effective web crawling algorithm that scours the internet for location updates from the web.
2. Utilize open source maps for obtaining up to date location data.
3. Utilize historical data patterns from ride-sharing data to detect POI changes.
4. Utilize a feedback mechanism that allows captains to report location discrepancies.

By pursuing these goals and objectives, the project aims to create a comprehensive solution for detecting outdated location data, ultimately improving the user experience and operational efficiency for these ride-sharing services.

2.5 Project Scope

The project encompasses two distinct phases, the research phase and the development phase.
The exploratory/research phase involves a comprehensive literature review to gather insights and best practices from existing research and solutions in the field. Additionally, the phase involves identifying open source data repositories that will serve as valuable resources for our project.
The second phase consists of the development stage. Our project aims to develop an automated location change detection system for Careem. To achieve this, a web crawler will be developed to scour the web for location information on different POIs. Additionally, a pattern detection algorithm will be developed using ride-sharing data from Careem to detect changes in POIs. Open source location data repositories such as OpenStreetMaps can be used to obtain location data of interest. Lastly, a driver feedback mechanism will be used to update the Careem POI database.
Keeping location data up-to-date involves two major processes: change detection and validation. Our project's focus is on automated change detection, but the scope does not include change validation through an algorithmic way. Hence, algorithm-based validation is left as future work.

2.6 Sustainable Development Goal (SDG)

In this project, we are focusing on two specific SDGs: Decent Work and Economic Growth, and Industry, Innovation, and Infrastructure.

2.6.1 Decent Work and Economic Growth

The first SDG is Decent Work and Economic Growth. By ensuring that location data is up-to-date and reliable, we contribute to the operational efficiency of these ride-sharing services, which, in turn, can lead to increased economic opportunities for drivers (Careem Captains) and the company as a whole. This project supports the goal of creating decent work by improving the livelihoods of individuals engaged in the gig economy. This is represented in figure 2.1

2.6.2 Industry, Innovation, and Infrastructure

The second SDG we are addressing is Industry, Innovation, and Infrastructure. Our hybrid solution leverages innovative techniques such as web crawlers, user feedback, and open-source maps to maintain accurate location data. By doing so, we are contributing to the development of more robust and technologically advanced infrastructure within the transportation and digital mapping sectors. This, in turn, can foster economic growth and innovation, aligning with the broader objectives of the SDG related to

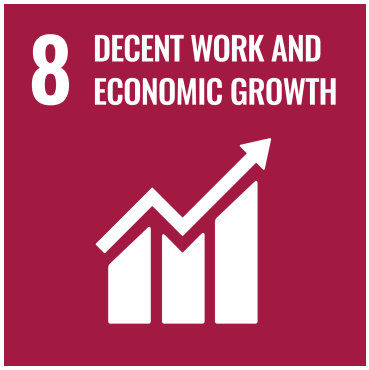


Figure 2.1: Decent Work Ethic Sustainable Development Goal

This figure represents the Decent Work Ethic SDG that is the target SDG of this FYP

industry, innovation, and infrastructure. Ultimately, our project represents a step toward ensuring that location-based services continue to evolve and meet the demands of the digital age. This is represented in figure 2.2

Figure 2.2: Industry Innovation and Infrastructure Sustainable Development Goal

This figure represents the Industry Innovation and Infrastructure SDG that is the target SDG of this FYP

Chapter 3 Literature Review / Related Work

This literature review chapter delves into the challenges posed by our problem domain, examining existing research on automated POI change detection and location data maintenance. The goal is to identify promising strategies and methodologies that can inform a comprehensive solution, ultimately improving the reliability and dynamism of location-based information for ride-sharing services.

3.1 Definitions, Acronyms and Abbreviations

Definitions of key terms along with expansions of abbreviations and acronyms are given below.

3.1.1 Definitions

Volunteered Geographic Information: Geospatial data and information collected and contributed voluntarily by individuals or communities

OpenStreetMap: A collaborative, open-source mapping platform that allows individuals and organizations to contribute, edit, and access geographical data and maps

World Wide Web: A global information system on the internet that allows users to access and interact with multimedia content and documents

Hyperlink: A clickable link that takes you to another webpage or resource online

Geoparser: A software tool or system that analyzes text and extracts geographical information, such as place names, coordinates, or location references, in order to identify and geographically link text-based data to specific locations on Earth

Geospatial: Data and information that is associated with specific geographic locations on the Earth's surface

Graph Neural Network: A type of artificial neural network designed to work with graph-structured data

IoU score: a commonly used metric in computer vision and object detection tasks that measures the overlap between two bounding boxes or regions of interest in an image

3.1.2 Acronyms and Abbreviations

ARI: Adjusted Rand Index

OSM: OpenStreetMap

NMI: Normalised Mutual Information

GNN: Graph Neural Network

GPS: Global Positioning System

VGI: Volunteered Geographic Information

DC3DCD: DeepCluster 3D Change Detection

IoU: Intersection over Union

SKPConv: Siamese Kernel Point Convolution

EFSKPConv: Encoder Fusion Siamese Kernel Point Convolution

3.2 Detailed Literature Review

In the following subsections, we delve into three distinct approaches for addressing the challenge of automating POI updates, namely, web crawling, obtaining open source location data and pattern changes detection. Image-based approaches and a comprehensive framework for data validation are explored as future prospects for solving our problem. Finally, existing commercial applications have been investigated to gain insight into practical solutions relevant to our problem domain.

3.2.1 Web Crawling

This section discusses three research items about web crawling and location data extraction, setting the stage for our project's efficient web exploration for location updates.

3.2.1.1 Summary of the research item

In the digital realm of the World Wide Web, the efficient extraction of location-based information is gaining increasing importance. This extraction heavily relies on web crawlers [1], the programs responsible for collecting web content by following hyperlinks. This literature review explores the significance of web crawlers in the realm of location-based information retrieval, with an analysis of existing work in this area.

One existing approach introduces an adaptive geospatially focused crawling strategy [2] that leverages Bayesian classifiers to prioritize links and pages of interest. Discussing the unique characteristics of geospatial information, the authors highlight the issue of its embedment in unstructured text, and the use of the Web's link structure in identifying geospatially relevant pages. Building on this knowledge, the authors propose the use of machine learning for predicting links in focused web crawling, and introduce a heuristic that prioritizes links based on their closeness to the last relevant page in a crawl branch.

In another method for creating a geospatial web search engine, a location-focused search architecture [3] is used. This method utilizes standard technologies like web crawlers and indexing systems, but enhances them with techniques that enable location-focused search. The authors use focused crawling techniques to find pages likely to be relevant to a specific location. To accomplish this, they guide a

geospatially focused crawler using a location classification derived from their geoparser. This geoparser can extract address-level location information from unstructured web pages, eliminating the need for metadata or other structured annotations.

Finally, a third method introduces a unique search engine specifically for location-based web searches [4]. This research outlines the structure of the search engine, which comprises a focused crawler, a location extractor, and an indexer. The focused crawler is engineered to fetch only pages that pertain to a specific topic. This is achieved by examining downloaded pages and only further processing those that align with the given topic as determined by a classifier. The location extractor's role is to pinpoint actual location data from web pages, while the indexer assigns a geographical context to pages, making them accessible for future spatial web searches.

3.2.1.2 Critical analysis of the research item

Adaptive geospatially focused crawling [2] presents an innovative crawling strategy for location-related web content. The results of their research indicate that their adaptive geospatial approach outperforms other strategies, with a harvest rate of approximately 12% over 1,000,000 downloaded documents within a 24-hour crawl, surpassing the baseline (4%) and link-distance heuristic (5.5%) strategies. However, further research is needed to evaluate the proposed solution's performance in different application scenarios and to address the limitations of the current study, such as the lack of information on missed relevant pages and the limited scope of the application scenario.

The second method, which focuses on extracting location data based on addresses from the web [3], provides an in-depth analysis of the process of extracting and verifying accurate addresses on German web pages. This is particularly useful for geospatial and location-based web searches. It highlights the potential of partial address matches in assessing linked pages' location and analyzing link structures. However, it acknowledges limitations such as focusing only on full addresses and the German web, and some location information may be missed. The parser performed well with a precision of 0.95 and a recall of 0.89, but the geocoding process translating addresses into geographic coordinates can lead to inaccuracies, with up to 7% results being in a different census tract.

Location-based web search [4] provides an in-depth analysis of the suggested techniques for web crawling and geoparsing, backed by experimental evidence. It validates the proposed seed selection methods through the evaluation of geographically focused crawling, achieving a precision rate of 85% and 90% for the regions of Oldenburg and Rügen respectively. Despite its effectiveness, the paper recognizes certain limitations. The focused crawler fails to retrieve all relevant pages, with an omission rate of 5-10% due to various typos or uncommon abbreviations that the location extractor can't always match. Furthermore, it's noted that the method is restricted to identifying geographic entities of a single building

address identified by its house number, excluding other forms of location information.

3.2.1.3 Relationship to the proposed research work

The existing research work in this domain, as outlined above, is closely tied to the creation and execution of an efficient web crawling algorithm aimed at retrieving location updates from brand websites and social media platforms on the internet. The existing work constitutes a fundamental element of the research, enabling the systematic acquisition of location data from online sources, aligning with our project's objectives.

3.2.1.4 Literature review summary table

Folllowing is the summary table for the above discussed literature review section.

Table 3.1: Web Crawling Literature Review Summary Table

This is a summary table of web crawling section 3.2.1 of literature review

Authors	Method	Results	Limitations
M. Kumar et al. [1]	Literature review on 248 studies from 1488 articles across 12 journals, and conferences.	The rise of mobile crawlers due to the Web's exponential growth.	Missed relevant articles due to ambiguity of keyword 'crawler.'
Dr. Ahlers et al. [2]	Geospatially focused crawling using Bayesian classifiers for location-relevant documents.	Outperforms other strategies with a 12% harvest rate (baseline: 4%, link-distance heuristic: 5.5%).	Recall measurement challenges due to the vast size of the web and non-sustainability of the initial performance peak.
Dr. Ahlers et al. [3]	Extraction and validation of precise addresses on German web pages using a validating parser.	Reliable extraction of address-level location information with a geoparser precision of 0.95 and recall of 0.89.	Geocoding introduces errors, with 50% of results in a nearby parcel and up to 7% attributed to a different census tract.
Dr. Ahlers et al. [4]	Location-based web search engine featuring a focused crawler, location extractor, and indexer.	Successfully identifies and indexes location data with a precision of 85% to 90%.	Omission rate of 5%-10% attributed to typographical errors or uncommon abbreviations.

3.2.2 Acquiring Open Source Location Data

This subsection highlights the importance of crowdsourced geographical data, specifically from OpenStreetMap (OSM) [5], in location-based research and applications, underscoring its reliability and versatility, and its relevance to our project.

3.2.2.1 Summary of the research item

In recent years, crowdsourced geographical information, known as Volunteered Geographical Information (VGI), has become a valuable resource in Geosciences, Earth Observation, and environmental sciences [6]. Notably, OpenStreetMap (OSM) stands out as a pioneering project in collaborative map creation. This literature review justifies the use of crowdsourced location data, particularly from OSM, for location-based research and applications, drawing insights from two key papers. The first paper [6] evaluates the quality and accuracy of OSM, focusing on London and England. It finds that OSM data has an average positional error of around 6 meters compared to traditional datasets, highlighting its reliability. OSM also demonstrated impressive growth, covering approximately 29% of England. However, about 4% of digitized lines lacked complete attributes, indicating a need for data enrichment. This study not only showcases OSM's potential but also its implications for VGI research.

3.2.2.2 Critical analysis of the research item

Volunteered Geographic Information (VGI) is a valuable resource for location data [6], particularly in areas where official datasets are lacking. It is a cost-effective and rapid method of data collection that encourages community engagement and citizen science. Its real-time updating capabilities make it ideal for applications requiring current information. However, the quality of VGI can be inconsistent due to the varying skills of non-professional contributors. Biases may emerge, favoring areas or populations with the resources and expertise to participate. There are also valid concerns about data privacy and security, and integrating VGI with official datasets can be challenging due to differences in formats and standards. Relevant statistics [6] from the paper illustrate the strengths and weaknesses of VGI. OpenStreetMap data displayed positional accuracy exceeding 70%, albeit with occasional drops to 20%. Completeness was found to be higher in urban areas compared to rural ones. Only approximately 40% of objects in OpenStreetMap met criteria for comparison with Ordnance Survey datasets. Quality varied depending on contributors, and errors did not distribute randomly geographically.

3.2.2.3 Relationship to the proposed research work

In recent years, crowdsourced geographical data has gained importance as a source of map data, and OSM stands out as a leading project in collaborative map creation. Given OSM's open-source nature and its merits, it serves as a valuable data source for our project. OSM's version history can be leveraged to detect location changes, which aligns with our proposed research work.

3.2.2.4 Literature review summary table

Folllowing is the summary table for the above discussed literature review section.

Table 3.2: Acquiring Open Source Location Data Literature Review Summary Table
This is a summary table of acquiring open source location data section 3.2.2 of literature review

Authors	Method	Results	Limitations
M. Haklay, [6]	Systematic analysis of VGI quality along with comparison of OSM data with Ordnance Survey datasets.	The spatial precision and completeness of OSM data varies based on the individual contributor and geographical area.	Possible biases within VGI and lack of an in-depth analysis of the data gathering and evaluation process.

3.2.3 Pattern Detection

This section reviews three innovative approaches for automated location change detection, offering valuable insights aligned with our project's goal.

3.2.3.1 Summary of the research item

In our rapidly evolving business landscape, companies often find it necessary to change their physical locations driven by factors such as market dynamics, cost optimization, expansion strategies, or even changes in customer demographics. These rapid relocations pose a problem for ride sharing platforms, because they have to employ human resources to keep their systems up-to-date. For an automated change detection and consequent updation, three approaches found in the literature pertaining to use of pattern detectors include; considering the ride-sharing data as a time series and detecting change using a structural pattern detector [7], forming a dynamic network of graphs for detecting changing locations via cluster-wise interactions and a statistical pattern detector [8] and performing change point detection in the dynamic network of ride-sharing via graph neural network [9].

Firstly, a novel approach to structural pattern recognition in time-series data [7] entails specialized structure detectors designed for generalized feature extraction, showcasing their effectiveness through experiments that achieve classification accuracies comparable to traditional statistical methods. This paper also discusses future research directions, including using structure detectors as cue generators and creating a taxonomy of time-series data. Overall, it offers a promising strategy for enhancing pattern recognition and knowledge acquisition in time-series data analysis.

Secondly, recognizing patterns can be translated to the realm of graphs and clusters, and this paper does the same by introducing a framework for clustering vertices from data and detecting change points in interaction intensities within dynamic networks, crucial for understanding temporal interactions [8]. It utilizes non-homogeneous Poisson point processes with cluster-dependent piecewise constant intensity

functions and common discontinuity points. Change in the intensity function for interaction of a specific cluster clearly indicates a change in the location of the POI. The paper states experimental results both on a generated dynamic network and a real world data set of cycle hire usage in London. It also demonstrates that the proposed algorithm outperforms baseline methods in terms of accuracy and computational efficiency, supported by the evaluation metric Adjusted Rand Index (ARI).
Lastly, another way of change-point detection in dynamic networks is by utilizing graph neural networks (GNNs), where identifying significant shifts in network structure or behavior is crucial [9]. The authors propose a novel approach combining clustering techniques and similarity analysis, using GNNs to capture temporal dynamics and node relationships within network snapshots. Their method is rigorously evaluated on both real-world financial data, successfully identifying structural breaks during major market events, and synthetic data, outperforming existing approaches in various change-point scenarios. The study underscores the importance of considering temporal dynamics in network analysis, particularly for abrupt changes, emphasizing GNNs' efficacy in addressing change-point detection tasks and their potential for analyzing non-stationary networks. These findings contribute valuable insights to the field of network analysis and the application of GNNs for understanding and predicting changes in dynamic networks.

3.2.3.2 Critical analysis of the research item

The literature reviewed for this subsection has a plethora of strengths but it also features some room for improvement. The thesis, "Generalised Feature Extraction for Structural Pattern Recognition in Time-Series Data," [7] introduces a modified approach to structural pattern recognition in time-series data through generalized feature extraction. The paper's strength lies in its clear presentation and experimental evaluation. However, it lacks a comprehensive discussion of potential limitations, implementation challenges, and dataset details. Additionally, a more detailed methodology explanation and a thorough comparison with existing approaches would enhance the paper's quality and comprehensiveness.
Where [7] lacks in a detailed methodology explanation, "Multiple change points detection and clustering in dynamic networks" [8] presents this feature as its strength. It puts forth and explains in great detail an interesting framework for clustering vertices and detecting change points in dynamic networks. The use of non-homogeneous Poisson point processes with cluster-dependent intensity functions is a novel approach to modeling the temporal dynamics of interactions. The paper's strength, as discussed earlier, lies in the thoroughness of the methodology employed. Additionally, the paper provides experimental results on both simulated and real data to validate the effectiveness of the methodology, demonstrating the ability of the framework to uncover both the cluster and temporal structure of dynamic graphs, in cases where other static or dynamic clustering algorithms struggle. However, there are a few limitations

of this work too. Firstly, the paper does not provide a detailed comparison with existing methods for dynamic network analysis. Furthermore, the experimental results are promising, but it would be beneficial to compare the proposed framework with other state-of-the-art approaches to assess its superiority. Overall, the paper presents a novel framework for clustering and change point detection in dynamic networks but requires some further comparative analysis with existing approaches and frameworks [8]. Lastly, the paper, ”Graph similarity learning for change-point detection in dynamic networks” [9] presents a fairly practical approach yielding good results evident from the findings stated. However, there are a few limitations to consider in this scholarly work too. Firstly, the study focuses primarily on financial data, which limits the generality of the findings to other domains. It would be valuable to explore the applicability of the proposed method in different contexts and datasets. Secondly, while the evaluation results demonstrate the superiority of the proposed method over existing baselines, it would be beneficial to have a more in-depth comparison and analysis of the performance metrics. Lastly, the study could benefit from a more detailed explanation of the underlying mechanisms and algorithms used in the proposed method. This would enhance the reproducibility of the research and facilitate further exploration and extension of the approach by other researchers.

3.2.3.3 Relationship to the proposed research work

In the context of the problem being tackled by our project, these literary works present distinct solutions, each of which is impressively unique in its innovative approach. The first approach encompasses obtaining ride-sharing data, transforming it into a dynamic network and performing structural pattern recognition on it [7]. The same time series data is used by [9] and pattern changes are detected and predicted via graph neural networks (GNNs). Additionally, with Careem’s ride-sharing data, non-homogeneous Poisson point processes (the customers and the points of interest) with cluster-dependent piecewise constant intensity functions and common discontinuity points done in [8] can be a suitable route for change detection. The change in the intensity function of a specific cluster is a clear indicator of a probable change in the Point of Interest. Implementing the framework from this paper will help us in capturing the temporal dynamics of interactions and make informed decisions for updating the locations in the ride-sharing platform. All of these approaches work towards solving Careem’s problem, enhancing the overall user experience and improving the efficiency of the platform’s operations.

3.2.3.4 Literature review summary table

Folllowing is the summary table for the above discussed literature review section.

Table 3.3: Pattern Detection Literature Review Summary Table

This is a summary table of pattern detection section 3.2.3 of literature review

Authors	Method	Results	Limitations
R. T. Olszewski [7]	Optimal piecewise fits for time-series data with structure detectors and curve-fitting algorithms.	Accuracies comparable to statistical counterparts by incorporating domain-specific characteristics.	Inconsistency in performance of structure detectors across different conditions.
M. Corneli et al. [8]	Framework for change point detection in dynamic network interaction intensities, using Poisson point processes.	Superior Adjusted Rand Indices for clustering and change point detection (0.98 and 0.96). Superior performance on real-world dataset.	The assumption of piecewise constant intensity functions, makes this approach less suitable.
D. Sulem [9]	Siamese graph neural network to detect change-points in dynamic networks by learning a data-driven graph similarity function.	F1-score of 0.95 on the synthetic SBM dataset and 0.92 and 0.89 on real-world correlation network datasets.	Poor system performance on dynamic networks with heterogeneous properties and nonlinear dynamics.

3.2.4 Future Work

This section explores potential extensions of our project that can be explored in the future. This includes image-based approaches for real-time location data acquisition, including 2D and 3D imagery, and discusses a methodology for assessing Point of Interest (POI) data quality, with potential applications in improving data validity and operational efficiency.

3.2.4.1 Image-based Approach

Image-based approaches offer potential solutions beyond traditional GPS coordinates for keeping location data up-to-date. This section discusses two such approaches based on 2D and 3D image data.

3.2.4.1.1 Summary of the research item

Location data isn't limited to GPS coordinates alone; it extends its reach into the realms of visual information, encompassing both 2D and 3D imagery. In the following research papers, we delve into innovative approaches that harness the power of images as a potent source for obtaining real-time location data.

The proposed approach for 2D image data [10] is based on three key stages: image matching, POI change detection, and aggregation. The authors use a feature-based approach for image matching, which involves extracting local descriptors from each image and matching them based on their similarity. For detecting changes in Points of Interest (POI), they employ a deep embedding that's trained with a triplet loss. This learns a similarity function in line with the goal of POI change detection. Subsequently, they

aggregate pairwise predictions across the entire venue to generate a ranked list of map locations where POI changes have been detected. This method can detect changes even when logos or signage are not clearly visible. It also holds the potential to ease the process of map maintenance, which is currently a manual, time-consuming, and costly task.

Transitioning from 2D image data, the DeepCluster 3D Change Detection (DC3DCD) method has been proposed for 3D images [11]. It is an unsupervised approach to detecting and categorizing changes at the point level in 3D point clouds. The method uses a deep learning-based approach, specifically a contrastive loss, to learn a feature representation that can distinguish between changed and unchanged points. The authors also propose a user-guided mapping of pseudo-clusters to real classes to better fit the use case. The method is evaluated on several datasets, and the authors show that it can achieve comparable results to fully supervised methods while requiring less annotation effort. The authors also analyze the behavior of the network during the training phase and propose future directions for research. Overall, the paper demonstrates the potential of unsupervised learning for change detection in 3D point clouds.

3.2.4.1.2 Critical analysis of the research item

The research presented under the 2D image [10] proposes a novel approach for detecting changes in Points of Interest (POIs) for proactive map updates. The authors provide a comprehensive benchmark of several alternative techniques for the image pair comparison step and report promising results. Specifically, the proposed method achieves an F1 score of 0.68 on the authoritative dataset, which outperforms the best alternative method by a margin of 0.08. However, the study has some limitations. Firstly, the proposed method relies on the availability of a large dataset of images captured in the same venue at different times, which may not always be feasible. For example, the authoritative dataset contains over 10,000 images captured in 6 different venues, which required significant effort to collect and annotate. Secondly, the authors acknowledge that their method may not work well for detecting subtle changes in POIs, such as changes in the menu of a restaurant. In fact, the proposed method achieves an F1 score of only 0.28 on the subset of POIs that underwent subtle changes, compared to an F1 score of 0.77 on the subset of POIs that underwent significant changes. Overall, despite these limitations, the proposed approach represents a substantial advancement in the field of proactive map updates and has the potential to improve the accuracy and efficiency of map content updates.

In the proposed 3D approach [11], the authors report that the DC3DCD method outperforms a fully supervised conventional machine learning algorithm that depends on hand-crafted features, achieving results comparable to fully supervised deep networks trained on 2.5D rasterization of point clouds. Per-class Intersection over Union (IoU) scores for several methods, including SKPConv and EFSKPConv,

which are fully supervised deep learning methods, have been provided. For example, EFSKPConv achieves an IoU score of 96.68% ± 0.30% for the "New building" class, while the DSM-Siamese method achieves an IoU score of 86.14% ± 0.65% for the same class. These statistics demonstrate the effectiveness of fully supervised deep learning methods for change detection in 3D point clouds, as well as the potential of unsupervised methods like DC3DCD to approach their performance. However, a notable limitation lies in the scarcity and complexity of acquiring high-quality 3D image datasets in real-world scenarios. This poses a significant challenge for the widespread applicability of the DC3DCD method in large-scale, real-world applications. Overall, the DC3DCD method offers a promising approach to unsupervised change detection in 3D point clouds, but further research is needed to address its limitations and improve performance.

3.2.4.1.3 Relationship to the proposed research work

In our pursuit of innovative approaches to obtain real-time location data, we recognize the invaluable role of image data in complementing traditional GPS coordinates. Since a quality dataset for 2D and 3D images is currently difficult to obtain, these image-based methodologies represent the next frontier in location data acquisition, beyond the current scope of our proposed project.

3.2.4.2 Data Validation

A methodology for assessing Point of Interest (POI) data quality, covering data collection, preprocessing, and quality metrics is discussed here.

3.2.4.2.1 Summary of the research item

The paper [12] describes the methodology used to evaluate the quality of POI data. Starting with data collection, the authors state collecting four POI datasets from different sources, including OSM [5] and OneMap [13]. These datasets were chosen because they represent different types of POI data sources, ranging from crowdsourced to commercial. After acquiring datasets, the authors perform data preprocessing on the collected datasets to remove duplicates, inconsistencies, and errors. They also standardized the attribute names and values across the datasets to facilitate comparison. Certain quality dimensions are then explained across which the datasets are evaluated. These quality attributes include completeness, positional accuracy, and thematic accuracy. These dimensions were chosen based on their importance for different applications of POI data. Furthermore, several quality metrics are identified for each quality dimension. For completeness [14], the metrics included attribute completeness, attribute uniqueness, and attribute value range. For positional accuracy, the metrics included spatial clustering, spatial distribution analysis, and attribute consistency. For thematic accuracy [15], the metrics included

category matching and attribute consistency.

Analysis of these datasets can be done by two different methods, intrinsic and extrinsic. The authors used both intrinsic and extrinsic methods to evaluate the quality of POI data. Intrinsic methods involve analyzing the internal statistics of the dataset, while extrinsic methods involve comparing the dataset with a reference dataset. The authors used a combination of both methods to provide a comprehensive evaluation of POI data quality. Afterwards, authors used a machine learning classifier to match POIs in the evaluated dataset with their corresponding counterparts in the reference dataset. The machine learning classifier was trained using a mix of hybrid sampling methods and ensemble learning algorithms to address the imbalance between the POI matches and non-matches in the training dataset. Finally, for evaluation of results, the authors evaluated the quality of POI data using the identified quality metrics and methods.

3.2.4.2.2 Critical analysis of the research item

This paper [12] conducts a comprehensive examination of approaches to validate POI data quality, categorizing them by data quality elements. It emphasizes the need for standardized validation metrics and acknowledges challenges in comparing diverse POI data sources due to limited documentation and proprietary workflows. The paper excels in its thorough literature review, encompassing various studies and review papers to cover a wide range of POI data validation approaches. Categorizing these methods based on data quality elements enhances our understanding of POI data assessment. Nonetheless, limitations include a lack of detailed analysis of how these validation methods perform on real-world Singaporean POI dataset. The paper also doesn't address potential biases in reviewed studies or provide actionable recommendations for improving POI data quality.

In summary, this paper provides a valuable overview of POI data validation approaches and highlights the need for standardization. However, practical implementation and further research are necessary to fully understand their effectiveness and address data quality challenges.

3.2.4.2.3 Relationship to the proposed research work

As an extension to the current project, this paper presents a use case in which the data manually being updated can be further passed through a process of validation, which is another human resource related problem for Careem. The company can use all or few metrics, presented in this work, which fulfill the semantic requirements of data validity essential for the company's smooth operations.

3.2.4.3 Literature review summary table

Folllowing is the summary table for the above discussed literature review section.

Table 3.4: Future Work Literature Review Summary Table

This is a summary table of future work section 3.2.4 of literature review

Authors	Method	Results	Limitations
J. Revaud et al. [10]	Three stages: image pair matching, POI change detection, and aggregation, with techniques for image pair comparison.	Deep embedding trained with triplet loss, achieves F1 score of 0.68 for image-pair evaluation and 0.62 for geographical evaluation.	Quality input photographs, the availability of image data, and the temporal distance between images.
I. de Gelis et al. [11]	Clustering the point clouds and employing a contrastive loss function to learn a similarity between them.	Urb3DCD-V2-1: DC3DCD - 87.5% acc, 0.63 IoU (no hand-crafted features), 89.04% acc, 0.68 IoU (with). AHN-CD: DC3DCD - 83.5% acc, 0.57 IoU; k-means - 80.5% acc, 0.49 IoU.	Assumptions about changes in point clouds, potential bias from user-guided strategy, computational complexity.
L. W. Yeow, et al. [12]	Reviewing POI validation methods on real-world datasets, training a machine learning classifier, and evaluating model's accuracy.	Hybrid sampling and ensemble learning trained a machine learning model. Accuracy 97.6%, balanced accuracy and overall accuracy on 97.2%.	Reference dataset quality. Not all POI data types may suit the machine learning classifier.
J. Revaud et al. [14]	Four methods to assess VGI quality: lineage analysis, internal spatial relations, data matching with authoritative references, and cross-validation with other VGI sources.	POI quality - Foursquare, Wikimapia (60% name, 50% category match) lower than OSM (80% name, 80% category match).	OSM dataset's heterogeneity poses challenges for POI comparison and quality assessment.
S. Meek et al. [15]	Creating a workflow supported by an ontological description to assess geolocated data from non-authoritative sources.	Framework can be used to dynamically optimize the usability value of the data captured.	The framework doesn't tackle privacy, security, and ethical concerns, which could be crucial for certain uses of crowdsourced data.

3.2.5 Related Applications

This section reviews three applications relevant to our project, providing insight into practical solutions to our problem domain.

3.2.5.1 Summary of the research item

Lyft, Strava, and Grab are three applications that utilize OpenStreetMap (OSM) for their base map data. Lyft [16] has developed an innovative driver localization algorithm [17] that identifies inaccuracies in maps to generate highly precise maps using OSM and real-time data. This algorithm collects crucial data such as missing roads, incorrect turn restrictions, one-way directions, destination signs, and other information that may be incorrect, omitted, or outdated in OSM. Additionally, Lyft has initiated a pilot program where a select number of rides equipped with car-mounted cameras gather data. This allows the company to update its maps with real-time changes such as road closures, construction work, or other obstacles.

Strava [18] leverages OSM along with its unique heat/popularity data to create a dynamic visual experience for maps [19], and to inform routing decisions and highlight points of interest on the map. Strava periodically fetches updated versions of OSM and then conducts additional processing to transform it into a routable basemap. The key aspect here is how Strava maps activity data onto OSM street and trail segments. This is particularly crucial for detecting and updating location changes. As users move and their activities are recorded, this data is mapped onto the corresponding OSM segments. This allows Strava to keep track of popular routes, detect changes in user movement patterns, and update the map accordingly.

Grab [20] integrates OSM for location change detection and updates [21]. While Google Maps serves as the foundational layer, Grab supplements it with its proprietary data. Grab has also introduced GrabMaps, an enterprise service offering location-based intelligence across 7 of the 8 countries it operates in. This service leverages OSM for detecting changes and updating location information. Grab collaborates with GlobalLogic, a software outsourcing firm, which has made numerous edits in recent months. These edits, facilitated by OSM, have allowed Grab to overwrite and update location data originally created by OSM members.

3.2.5.2 Critical analysis of the research item

Lyft, Strava, and Grab have all leveraged OSM in innovative ways to enhance their services, but each has faced unique challenges.

Lyft's implementation of a driver localization algorithm [17] for real-time map error detection and updates is praiseworthy. However, the method of relying on drivers for data gathering could potentially lead to bias or inaccuracies. Additionally, while the concept of using vehicle-mounted cameras for data collection is forward-thinking, it not only brings up issues related to privacy but also hinges on the reach of these camera-equipped vehicles.

Strava's approach of combining OSM with its own unique data for map visualization and routing de-

cisions is a good example of leveraging multiple data sources. However, their reliance on periodically pulling updated versions of OSM for processing could lead to lags in reflecting real-time changes. Moreover, the issue of data mismatch when updating to a more current OSM basemap can cause inconsistencies [22] and requires users to re-submit export requests of historical data downloads.

Grab's approach of using Google Maps as the base and adding its own information provides a comprehensive mapping solution. However, their collaboration with GlobalLogic led to numerous incorrect edits that overwrote information created by OSM members. This not only led to inaccuracies but also raised concerns about the respect for the work of OSM volunteers. The suspension of work by the GlobalLogic team in Thailand [23] due to these issues highlights the challenges Grab faces in maintaining map accuracy.

3.2.5.3 Relationship to the proposed research work

The experiences of Lyft, Strava, and Grab provide valuable insights for our project on using OSM and crowdsourced location data to detect location changes.

Lyft's driver localization algorithm [17] that detects map errors could serve as a starting point for developing a similar mechanism in our project to detect and validate location changes. Their method of collecting real-time data through car-mounted cameras could also be adapted to Careem's needs, by encouraging Careem captains to contribute real-time location data.

Strava's technique of overlaying activity data on OSM street and trail segments could be particularly useful for Careem's system that relies on OSM for detecting and updating location changes. This method, which involves tracking movement or activity in specific locations, could enhance the system's ability to monitor and respond to changes in real-time. Furthermore, Strava's practice of regularly fetching updated versions of OSM for processing could be incorporated into the system, ensuring that the location data remains up-to-date, reflecting the most recent changes and updates from OSM.

Grab's journey underscores the significance of valuing the contributions of OSM volunteers for Careem's system that uses OSM for location change detection and updates. It's vital to ensure that any additions or modifications to the map enhance its precision and utility. This is an essential factor for our system as it depends on crowdsourced location data. Respecting and acknowledging the work of OSM contributors not only maintains the integrity of the data but also ensures the continuous improvement and accuracy of location updates.

In summary, these applications demonstrate the potential and challenges of using OSM and crowdsourced data for location-based services. Their experiences can guide the design and implementation of Careem's location detection system to ensure it is effective, respectful of the OSM community, and provides value to its users.

3.2.5.4 Related applications summary table

Folllowing is the summary table for the above discussed related applications.

Table 3.5: Related Applications Summary Table

This summary table is for the related applications' summary, discussed in the literature review section 3.2.5

Application	**Features**	**Relevance to Application**	**Limitations**
Lyft [16]	Driver Localization Algorithm, Real-Time Map Error Detection and Updates	Identifies inaccuracies in maps to generate highly precise maps using OSM.	Relying on drivers for data gathering could potentially lead to bias or inaccuracies, as well as raising privacy concerns
Strava [19]	Routable Basemap, Dynamic Visual Experience for Maps	Periodically fetches updated versions of OSM and then conducts additional processing to transform it into a routable basemap.	The issue of data mismatch when updating to a more current OSM basemap can cause inconsistencies
Grab [20]	Integration of OSM, GrabMaps	Grab uses OSM for detecting and updating location changes, supplementing Google Maps with its proprietary data.	Partnership with GlobalLogic resulted in numerous inaccurate edits to OSM, raising concerns about respect for OSM volunteers' work.

Chapter 4 Software Requirement Specifications

In this chapter, we will discuss the GeoSync system's functionality and expected performance. The system's requirements and use cases will also be covered.

4.1 List of Features

The major features of the GeoSync system are listed in this section. They are listed below:

- OSM Data Collection
- Location Data Repository Obtained Using Web Crawling
- Real time Use of Web Crawling
- Pattern Detection to identify location changes
- Driver Feedback Mechanism

4.2 Functional Requirements

The functional requirements of our system are listed in this section.

4.2.1 OSM Data Collection

The system shall retrieve OSM data for specified geographic regions. It should support the collection of data for different types of OSM elements including nodes.

4.2.2 Location Data Repository Obtained Using Web Crawling

The application must employ web crawling techniques to extract and gather location data from various online sources. It should store the collected data in a centralised repository for easy access and management.

4.2.3 Real-Time Use of Web Crawling

The system should check for updated location data in real-time using web crawling whenever a ride is requested. It must implement mechanisms to detect and retrieve new or modified data from online sources promptly.

4.2.4 Pattern Detection to Identify Location Changes

The application should utilize pattern detection algorithms to identify changes in location data using the ride-sharing data.

4.2.5 Driver Feedback Mechanism

The system shall provide a feedback mechanism for drivers interacting with the app. It should allow drivers to submit feedback on location accuracy, completeness, and any discrepancies encountered.

4.2.6 Notification System

Set up a notification system to alert users about significant changes in drop-off location data obtained from the change detection modules.

4.3 Quality Attributes

The quality attributes of the GeoSync system are as follows:

- Performance
- Reliability
- Availability
- Reusability

4.4 Non-Functional Requirements

The non-functional requirements for our system are discussed in this section.

4.4.1 Performance

- The system shall be able to detect location changes in real time.
- The system should be able to accommodate real time execution of the detection modules to detect change in POI locations when a ride is booked.

4.4.2 Readability

- The system should be able to consistently detect location changes accurately, without any errors or crashes.

4.4.3 Availability

- The system should remain available 24/7
- Drivers should be able to access driver feedback mechanism at all times

4.4.4 Reusability of Change Detection Modules

- Change detection is broken down into different modules based on different approaches
- Modules for change detection should be reusable in different applications

4.5 Assumptions

Following are some assumptions made for the system.

- A majority of the online sources for location data permit web crawling and data extraction.
- Patterns related to user activity exist in the ridesharing dataset that can be used to detect POI changes.
- Drivers are able to consistently provide accurate and up-to-date location data information.
- OSM data is the most up-to-date open source available for POI data.

4.6 Use Cases

This section lists the major use cases of the system.

4.6.1 Sign Up

Name	Sign up		
Actors	Customer, Driver		
Summary	The user enters account details on the signup page and a new account is created.		
Pre-Conditions	The database does not have an existing account with the same account details.		
Post-Conditions	A new account is successfully created by the system and the user is directed to the login page.		
Special Requirements	None		
Basic Flow			
Actor Action		**System Response**	
1	The user opens the signup page.	2	The system prompts the user for new account details

3	User inputs valid credentials.	4	The system verifies that an account with matching credentials does not exist.
		5	The system creates a new account and directs the user to the login page.
Alternative Flow			
3	The user inputs invalid credentials	4-A	The system displays an error message.
Alternative Flow			
3	The user inputs credentials of an already existing account.	4-B	The system displays an error message.

4.6.2 Login

Name	Log in		
Actors	Customer, Driver		
Summary	The user fills the login credentials on the login page and is authenticated by the system.		
Pre-Conditions	The user has an existing account and is not already logged in.		
Post-Conditions	The user is successfully authenticated by the system and is logged into their account.		
Special Requirements	None		
Basic Flow			
Actor Action		**System Response**	
1	The user opens the login page.	2	The login page is displayed asking for credentials.
3	User inputs valid credentials.	4	The system authenticates the user and directs them to the home page.
Alternative Flow			
3	The user inputs incorrect credentials.	4-A	The system displays an error message.

4.6.3 Detect Location Change

Name	Detect Location Change		
Actors	Customer		
Summary	Any changes in the drop off location provided by the customer, which happened in the real world and is not reflected in the system will be detected at the start of the ride.		
Pre-Conditions	Customer is logged in.		
Post-Conditions	Any changes detected are shown by a notification to the customer.		
Special Requirements	None		
Basic Flow			
Actor Action		**System Response**	
1	The user requests a ride.	2	The system prompts for a drop-off location.

3	The user enters the drop-off location.	4	The system runs the detection modules to detect any change in drop-off location.
		5	Change in the location is detected.
		6	Go to Use Case Notify User.
Alternative Flow			
		5-A	No change in drop-off location is detected.
		6-A	No action is taken.

4.6.4 Notify User

Name	Notify User		
Actors	Customer		
Summary	The customer is notified of the change detected in their drop-off location.		
Pre-Conditions	A change in drop-off location is detected by the detection modules and new notification pops up.		
Post-Conditions	The user has been notified of the change in drop-off location.		
Special Requirements	None		
Basic Flow			
Actor Action		**System Response**	
1	The user clicks on new notifications.	2	The system shows details of location change.

4.6.5 Provide Feedback

Name	Provide Feedback		
Actors	Driver		
Summary	The driver is prompted for feedback regarding accuracy of location data at the end of the ride.		
Pre-Conditions	The driver's ride has ended		
Post-Conditions	The location discrepancy count for that drop-off location is incremented in the system database.		
Special Requirements	None		
Basic Flow			
Actor Action		**System Response**	
1	The user ends the ride.	2	The system shows a feedback prompt regarding drop-off location accuracy.
3	The user gives feedback that location discrepancy exists.	4	The system records user feedback in the database.
Alternative Flow			
3-A	The user gives feedback that no location discrepancy exists.	4-A	The system does not record user feedback in the database.

4.7 Hardware and Software Requirements

To develop and deploy the project, hardware and software requirements needed are being discussed in this section.

4.7.1 Hardware Requirements

The hardware requirements to successfully use the system at the user end include:

4.7.1.1 Sufficient computational capacity

Sufficient computational capacity is required to run the web crawler, pattern detection module and driver feedback mechanism.

4.7.1.2 Memory in the system

The device shall have the sufficient space available for the location data repositories

4.7.1.3 Internet connection

A reliable internet connection is required for the system to maintain up-to-date location data and notify users of changes during a ride.

4.7.2 Software Requirements

The software requirements for the development of the system are as follows:

- Django
- Pycharm
- Tailwind CSS
- Google Colaboratory
- OSM APIs
- BeautifulSoup library
- Requests library
- MongoDB
- PostgreSQL

4.8 Graphical User Interface

The users of our system include drivers and customers. The graphical user interface is shown for different pages along with a description of their working.

4.8.1 Signup page

A new user can register by creating an account. The signup page prompts the user to enter a valid username, email address, phone number and password.

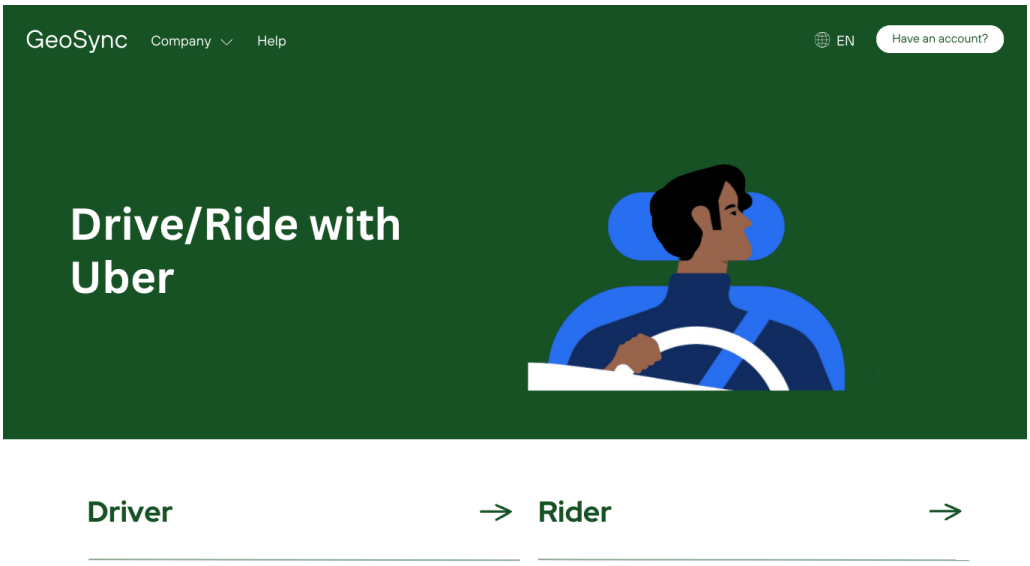


Figure 4.1: Signup Page

This figure represents the Signup page of the system

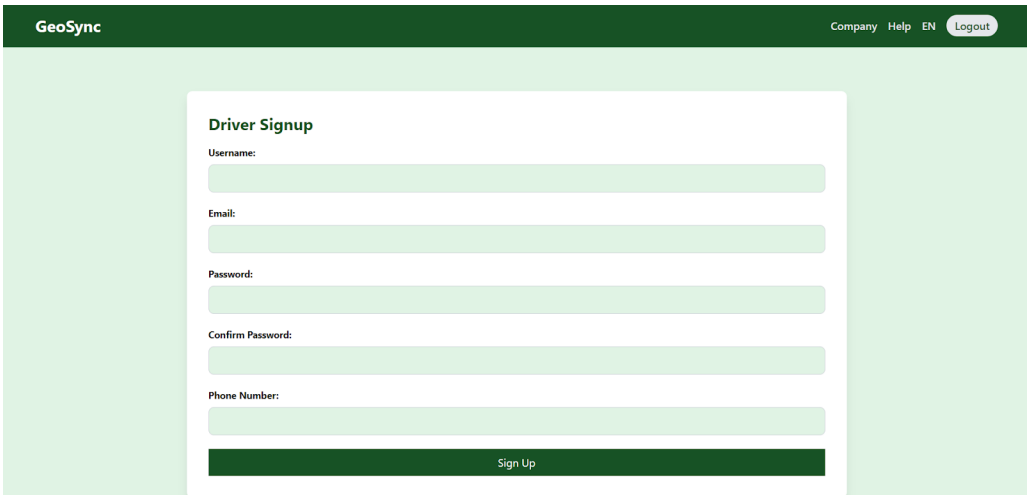


Figure 4.2: Rider Signup Page

This figure represents the Signup page for the riders in the system

4.8.2 Login page

The user can login using the login page. It requires the user to enter login credentials for either the user or driver account.

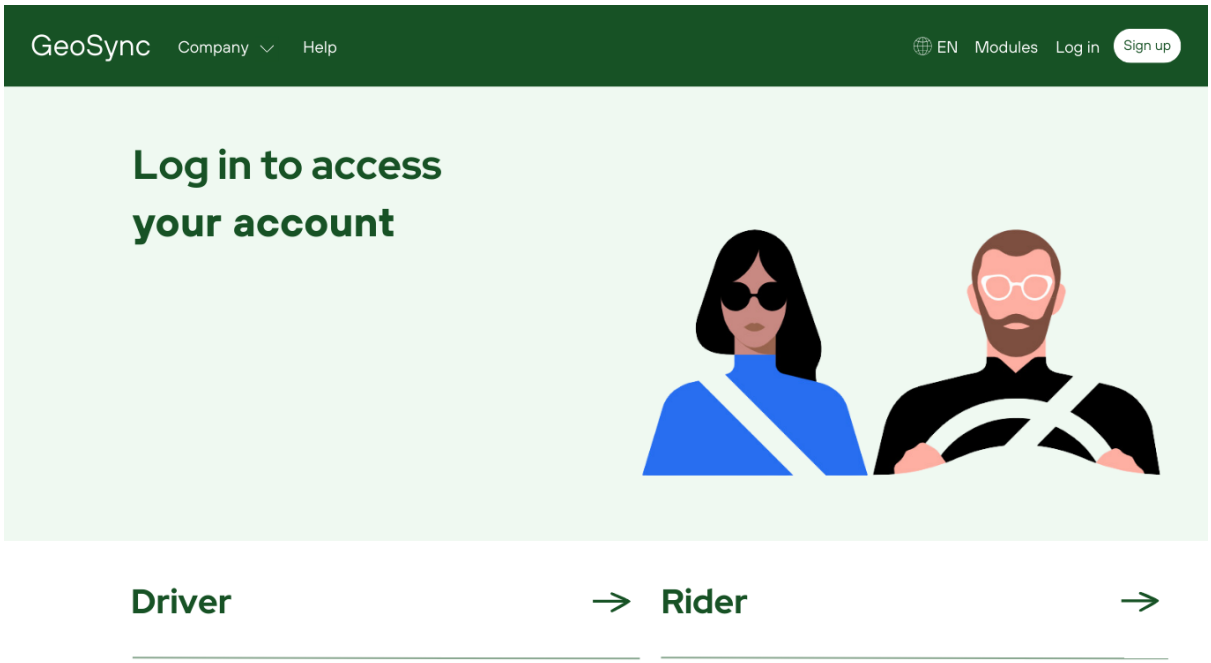


Figure 4.3: Login Page

This figure represents the Login page of the system

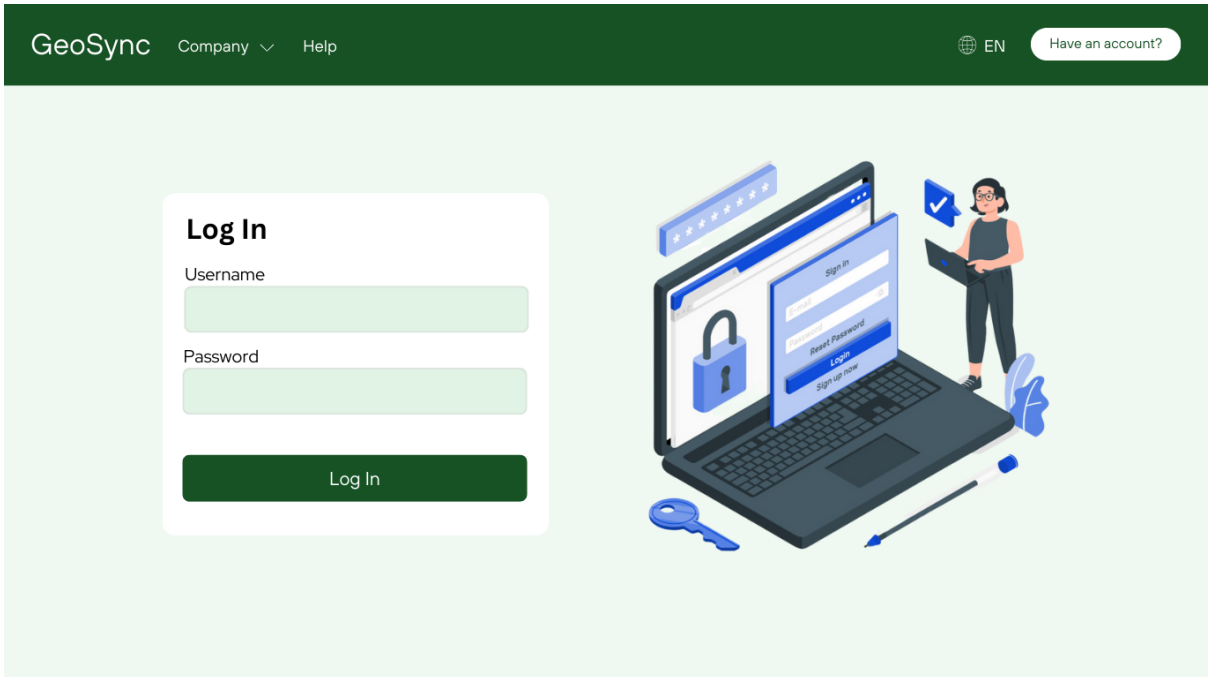


Figure 4.4: User Login Page

This figure represents the Login page for users of the system

4.8.3 Home Page

The homepage will appear after the user logs in. In this page, a user chooses the pickup and drop-off locations, followed by requesting a ride. The change detection modules are then run to detect any changes in the drop-off location.

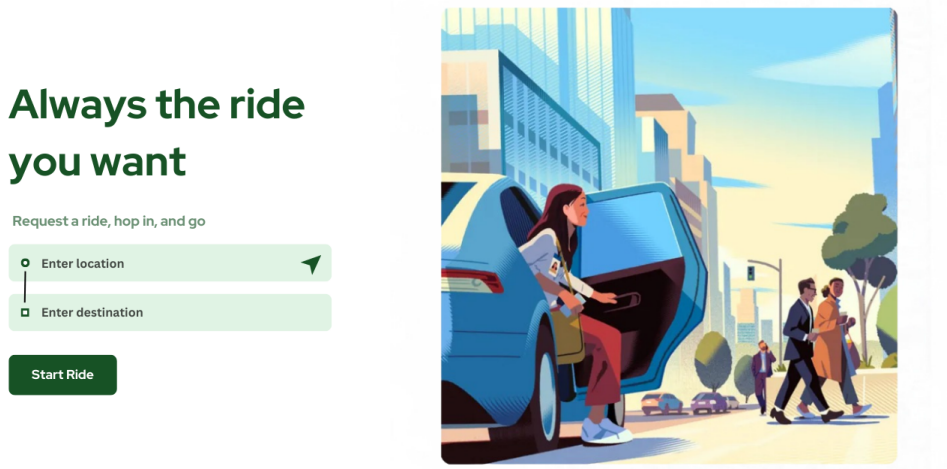


Figure 4.5: Home Page

This figure represents the Home page of the system where users can book a ride and see validity of their destination addresses

4.8.4 Notification Page

The notification page will appear after the change detection module is done running. In this page, the user is notified if the drop-off location they entered has changed.

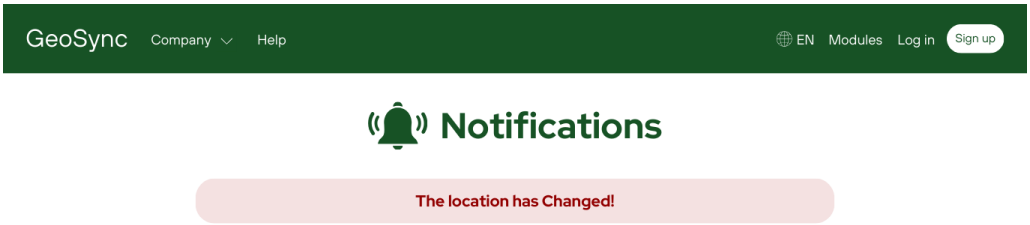


Figure 4.6: Notification Page

This figure represents the Notifications page of the system.

4.8.5 Driver Feedback Page

The driver feedback page is accessed by the Driver in order to report any discrepancies in the drop-off location during one of their rides.

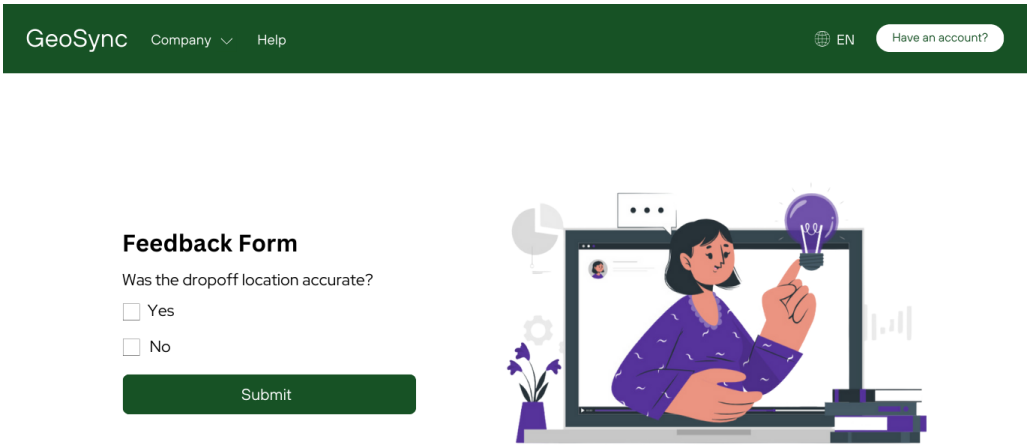


Figure 4.7: Driver Feedback Page

This figure represents the Driver feedback page of the system.

4.9 Risk Analysis

Some of the technical and business risks that may be encountered during system development are explained in this section.

4.9.1 Anti-Crawling Measures

Websites may implement anti-crawling measures, such as CAPTCHA or IP blocking, hindering the web crawling process and limiting access to essential data sources.

4.9.2 Algorithmic Complexity

Complex algorithms used in web crawling and pattern detection may result in increased processing times, affecting the real-time nature of location data updates.

4.9.3 Integration Challenges

Integrating multiple sources, such as web crawlers, open-source maps, and historical data, may lead to compatibility issues or data inconsistencies.

4.9.4 Dependency on External Platforms

Relying on external platforms, such as open-source maps, exposes the project to the risk of changes or disruptions in these platforms.

4.9.5 Feedback System Reliability

The effectiveness of the feedback system relies on the active participation of Careem Captains. Inconsistent or unreliable feedback may impact the accuracy of location data updates.

4.10 Conclusion

This chapter discusses the functional and non-functional requirements, along with other software and hardware prerequisites necessary for the development of our system. Furthermore, it provides an in-depth analysis of use cases, discussing their primary and alternative flows. Additionally, potential risks associated with this project are also examined.

Chapter 5 Proposed Approach and Methodology

The GeoSync project consists of the acquisition of open source location data and three distinct modules for POI change detection: pattern recognition, web scraping and driver feedback system. This section provides a detailed explanation of how each module within the GeoSync system operates. It delves into the specifics of each module's definition, its significance within the system, its functionality, and any algorithms employed for its operation.

5.1 Acquisition of Open Source Location Data

This section focuses on obtaining location data from crowdsourcing. The GeoSync system requires a constant source of location data which is up-to-date.
OSM has established itself as a dependable provider of location data, featuring a planet file for map data and supported by numerous open-source applications. Among these, we have identified Osmosis as an effective tool for handling OSM data, which will be utilized to:

- Create data dumps from an existing database
- Import data dumps into a new database
- Generate modifications using the history tables in the database
- Compare two data dump files and generate a change set
- Apply changesets to a local database
- Reorganise the information within the data dump files

The figure below represents how Osmosis can be used to process and handle OSM data.

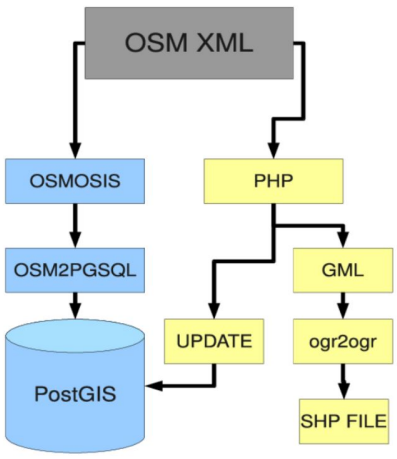


Figure 5.1: OSM Data Collection Strategy

Flowchart of processing applied to the OSM XML file.

5.2 Change Detection

This section focuses on three distinct change detection modules. It provides a detailed explanation of how each module within the GeoSync system operates.

5.2.1 Pattern Detection

This section details the use of a data-driven graph similarity function facilitated by a Siamese Graph Neural Network (GNN) architecture, Sort-k pooling layer, and a sliding window approach.

5.2.1.1 Data Preprocessing

A temporal sequence of graph snapshots, where each snapshot represents the network structure at a specific time point is obtained from the unprocessed ride-sharing data.

5.2.1.2 Model Training

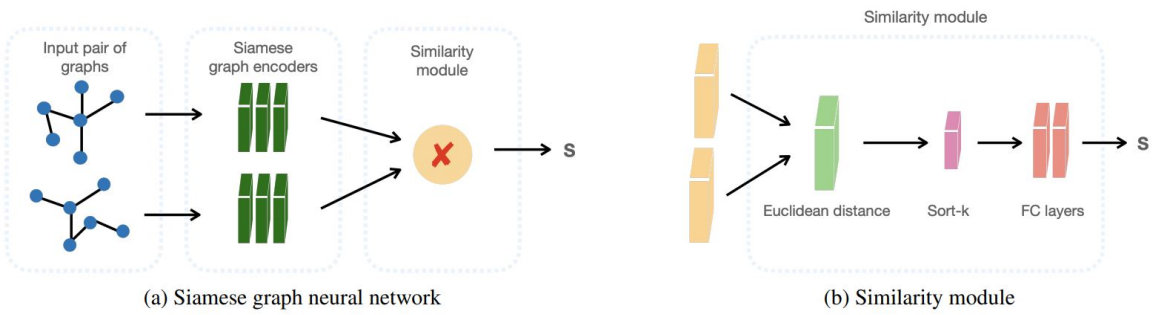
A Siamese GNN architecture comprising two identical GNNs with shared weights is trained on the pairs of graphs from the time series data prepared in preprocessing step. This network's output is fed to a similarity module. The following steps are involved in similarity module:

- The **Euclidean distance** module is used to compute the pairwise distances between the node-level embeddings for the two input graphs. This distance measure captures the structural differences between the two graphs, as nodes that play distinct structural roles in the two graphs will have larger distances between their embeddings.
- The **Sort-k pooling** layer selects the top-k most similar nodes from each graph and constructs a new graph with nodes connected by the edges weighted by their similarity scores
- Output from the Sort-k pooling layer is then fed into **FC** (fully connected layers) to produce the final similarity score.

For training, Random Scheme is used. We create pairs of graphs from the training sequence and label them based on the presence of change points. The dataset is uniformly sampled to include an equal number of positive and negative samples. The number of pairs is a hyperparameter, chosen as $10 \times T$ where T is the number of timestamps in the training sequence of graph snapshots.

5.2.1.3 Validation and Performance Evaluation

For validation, Sliding Window Scheme is used. We form pairs of graphs from the network sequence within a specified temporal window. Label pairs using the same procedure as in the Random scheme



(a) Siamese graph neural network (b) Similarity module

Figure 5.2: Siamese GNN Module Design

Architecture of our graph similarity learning model. The general pipeline (a) is a siamese GNN where the output module is a similarity module (b). We design the latter with Euclidean distance, Sort-k pooling operations, and fully-connected layers, for measuring the proximity of snapshots in dynamic networks.

and validate the model based on this data subset. Finally, performance of the model will be evaluated using localisation error and adjusted F1 score.

5.2.2 Web Crawler

This section explains how a web crawler will be used to extract location data from online sources and convert the extracted location information into geocoordinates.

5.2.2.1 Location Data Extraction

In the initial phase of the project, the target company or brand websites containing location information will be identified and their structure understood. Python-based tools like BeautifulSoup, Scrapy, and Selenium will be used to develop a web crawler to navigate the sites and extract location information.

5.2.2.2 Geocoding

The extracted location information could be in the form of addresses or place names. A geocoding service such as Google's Geocoding API or OSM's Nominatim service will be used to convert the extracted location information into latitude and longitude, which will be stored in a repository.

5.2.3 Driver Feedback Mechanism

This section discusses how the driver themselves can play a crucial role in location data up-to-date using a feedback mechanism.

5.2.3.1 End of Ride Prompt

Whenever a ride ends, the driver will be shown a prompt on their screens asking if the location the customer chose was accurate in the system's location data. The discrepancy count for that location will

be incremented in the database if a change is reported. If multiple drivers report discrepancies between the customer's chosen location and a certain POI, that is an indication of an inaccurate location in the system's location data.

5.2.3.2 Reporting Location Discrepancies

In addition, the driver will always have the ability to report location discrepancies that they observe in their regular day to day routine. Since drivers have more knowledge about the true location of POIs, their knowledge will be harvested by giving them the option to report location discrepancies whenever they see one. Their reports will be saved in the system and will also be used as a threshold to determine if a certain POI location data is outdated.

5.3 Conclusion

This chapter discussed the working of the individual modules of the Geosync system. When a drop-off location is entered by a user when booking a ride, the system will run the change detection modules, and the user will be notified if a change is detected.

Chapter 6 High-Level and Low-Level Design

In this section, we will discuss high-level and low-level designs of the GeoSync system.

6.1 System Overview

The GeoSync system aims to detect and notify users of location changes in real time when a user enters a drop-off location when booking a ride on the application. The users of the system will be customers who can login to the system and book a ride, as well as drivers who can report location discrepancies that are recorded by the system. Change in drop-off location is detected by running the change detection module whenever a ride is booked and drop-off location is added by the user.

6.1.1 Change Detection Module

The GeoSync system achieves reliable and accurate change detection through the seamless integration of three distinct components within its change detection module. The design of the three modules is discussed in the following subsections.

6.1.1.1 Pattern Detection

This module outlines a methodology for employing a data-driven graph similarity function with a Siamese Graph Neural Network (GNN). Temporal graph snapshots from ride-sharing data are processed through the Siamese GNN, employing Euclidean distance and Sort-k pooling to capture structural differences and construct a new graph. The training process incorporates a Random Scheme, generating labeled pairs based on change points. The result of this module determines whether or not a change exists in the drop-off location provided by the user.

6.1.1.2 Web Crawler

This module outlines the utilization of a web crawler for extracting location data from online sources and converting it into geocoordinates. The initial phase involves identifying target company or brand websites and developing a web crawler for extracting location information. Subsequently, in the geocoding phase, the extracted location data is processed to obtain latitude and longitude coordinates.

6.1.1.3 Driver Feedback

This module introduces a Driver Feedback Mechanism for ensuring location data accuracy. After each ride, drivers receive prompts to confirm customer-chosen locations, updating the database with discrep-

ancy count for those locations. Drivers can also actively report location issues during their routines, utilizing their firsthand knowledge. These reports serve not only for real-time updates but also establish thresholds to identify outdated POI data, ensuring a dynamic and reliable system.

6.2 Design Considerations

This section addresses critical aspects that shape the foundation of the design process for the GeoSync system.

6.2.1 Assumptions and Dependencies

This section delves into the foundational assumptions and dependencies that shape the GeoSync system's design and functionality.

6.2.1.1 Assumptions

- The system assumes continuous availability of OpenStreetMap (OSM) data for Point of Interest (POI) information. This assumption is based on the expectation that OSM, as an open-source platform, consistently provides up-to-date and accessible geographic data.
- GeoSync assumes that a majority of online sources for location data permit web crawling and data extraction. This is essential for keeping the system's geographic information current and comprehensive.
- Assumption that discernible patterns related to user activity exist in ridesharing datasets. These patterns are crucial for effective detection of changes in Points of Interest (POIs) and form the basis for pattern detection algorithms.
- The design assumes that drivers consistently provide accurate feedback.

6.2.1.2 Dependencies

- The system is dependent on web crawling techniques to extract and gather location data from various online sources. This dependency underscores the importance of a reliable web crawling mechanism for maintaining an up-to-date location repository.
- The system is dependent on continued availability and reliability of OSM data. Any disruptions or changes in the OSM data structure can impact the system's ability to retrieve accurate and relevant geographic information.

- Dependencies exist on the existence of discernible patterns in ridesharing datasets for effective change detection.
- The system relies on consistent contributions from drivers through the feedback mechanism.

6.2.2 General Constraints

GeoSync's design is shaped by various global constraints, each significantly influencing the system's software architecture and functionality.

6.2.2.1 Hardware Constraints

The computational capabilities of laptops, referred to as, impose limitations on processing power. This constraint necessitates the optimization of data processing algorithms to ensure the system's efficiency on devices with moderate computing resources.

6.2.2.2 Web Crawling Resource Availability

The reliance on web crawling for location data extraction introduces constraints related to the availability and volatility of online resources. The design must accommodate variations in online source accessibility and data structure changes.

6.2.2.3 Performance Requirements

Constraints related to performance expectations, such as real-time location change detection, influence the system's architecture. Meeting performance requirements is crucial for delivering a responsive and efficient user experience.

6.2.3 Goals and Guidelines

Goals and guidelines guiding the design and development of GeoSync are discussed below:

6.2.3.1 Efficiency and Simplicity

Adhering to the KISS principle ("Keep it simple, stupid!") is a primary goal. The emphasis on simplicity aims to enhance user experience and facilitate ease of maintenance.

6.2.3.2 Optimised Resource Utilisation

Prioritising efficiency over excessive resource consumption aligns with the goal of making GeoSync accessible and responsive across a diverse range of computing environments.

6.2.3.3 User-Centric Design

Design decisions aim to ensure that the system's functionality and interface resonate with end-users, providing a familiar and intuitive experience. Moreover, the system aims to keep the computationally expensive modules for change point detection running in the background transparent in terms of time by optimised resource utilisation to enhance user experience.

6.2.4 Development Methods

The software design and development for GeoSync adopts an agile development approach. While no single formal or published method is strictly followed, principles from agile methodology, Scrum, influence the development process. This approach allows for flexibility and responsiveness to evolving and fuzzy requirements along with keeping the client, Careem in our case, in the loop. Various design methods including iterative and evolutionary were considered, but due to Scrum's better capability of ensuring retrospective and introspective work paired with the possibility of active client involvement, which in the context of this project Careem exhibits, and fuzzy requirement handling made it the selected candidate.

6.3 System Architecture

This section outlines the architecture of the GeoSync system through diagrammatic representations. It provides a high-level overview of the system's functionality, as well as detailing the decomposition of responsibilities among subsystems. The high-level system architecture of GeoSync comprises two main components: the open source location data acquisition module, the POI change detection module. The open source location data acquisition module is responsible for collecting relevant geographical information. The POI change detection module consists of three sub-modules: pattern recognition, web scraping, and a driver feedback system, which collectively identify and notify users of real-time location changes when a ride is booked. The driver feedback module enables drivers to report location discrepancies, which are recorded in the database. The seamless integration of these components ensures the system's effectiveness in detecting drop-off location changes, providing a robust and user-friendly experience for both customers and drivers. System architecture is present in the figure 6.1.

6.3.1 Pattern Detector

The depicted Pattern Detector subsystem architecture comprises a data preprocessor, Graph Neural Network (GNN), similarity module and model trainer. The process begins with data preprocessing, where a temporal sequence of graph snapshots is derived from unprocessed ride-sharing data. The Siamese

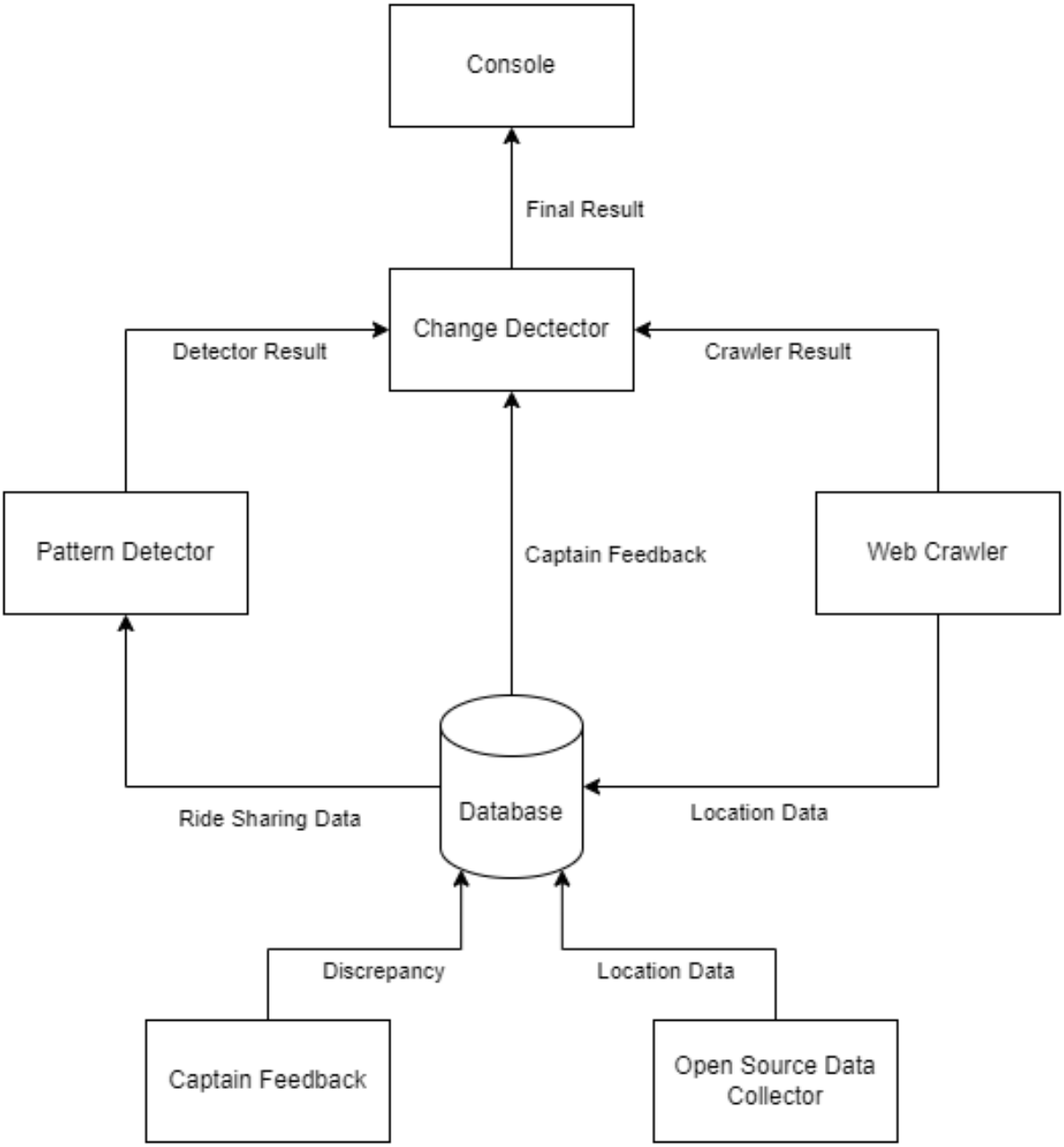


Figure 6.1: High-level Architecture Diagram

This is a high-level architecture diagram of GeoSync.

GNN is then trained on pairs of graphs from the time series data. The similarity module is composed of three steps- Euclidean distance computation, Sort-k pooling layer and fully connected layers- to produce the final similarity score. For the model training module, a Random Scheme is employed, generating pairs of graphs labelled based on change point presence. All these sub-modules are used by the Pattern Detector subsystem to detect location changes. Structure of this module is in figure 6.2.

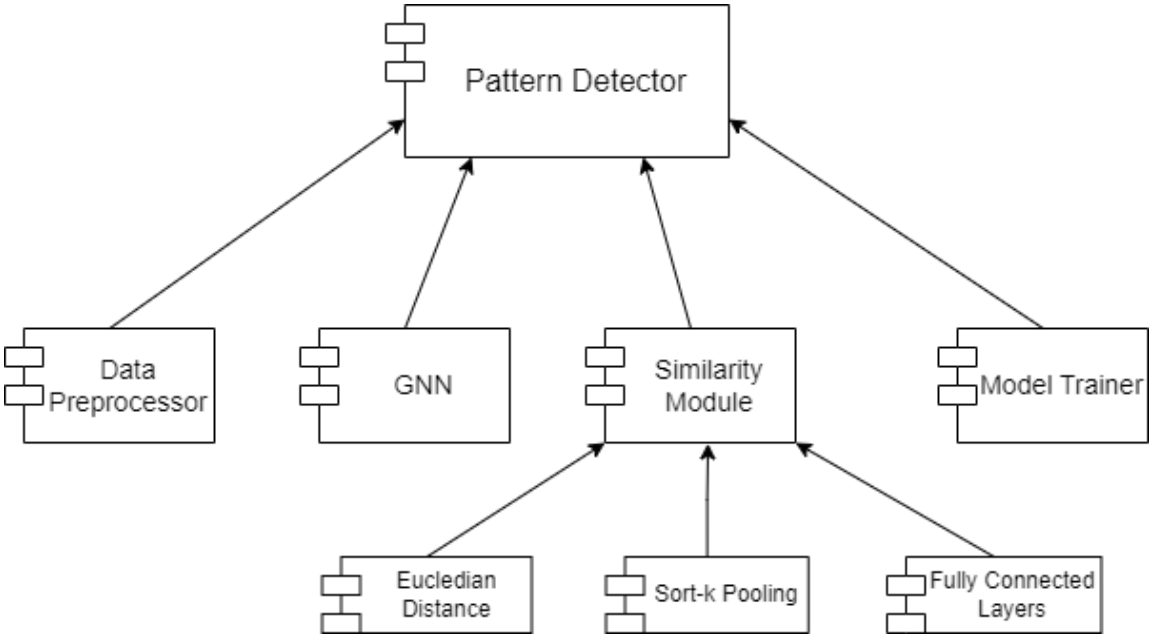


Figure 6.2: Pattern Detector - Subsystem Architecture Diagram

This is a high-level architecture diagram of GeoSync.

6.3.2 Web Crawler

The Web Crawler sub-module of the system serves the purpose of acquiring location data from identified company or brand websites. This subsystem encompasses two key components: Location Data Extraction and Geocoding. The location extractor employs web crawling and web scraping techniques to navigate web pages and extract relevant location information. This data, which may consist of addresses or place names, undergoes a subsequent geocoding process, where the extracted location data is converted into latitude and longitude coordinates Structure of this module is in figure 6.3.

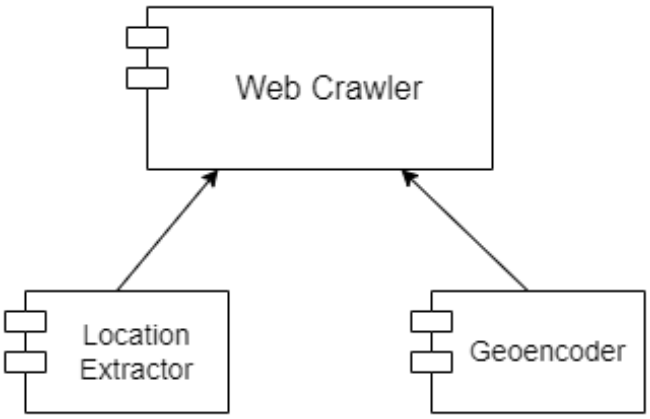


Figure 6.3: Web Crawler - Subsystem Architecture Diagram

This figure represents the Web Crawler Subsystem

6.3.3 Driver Feedback Module

The driver feedback subsystem has two major components- the feedback prompter and discrepancy reporter. The feedback prompter displays a feedback form to drivers to report any inaccuracies in the drop-off location information stored in the system. The discrepancy reporter will then be used to increment the discrepancy count for that particular location in the database in case a change is reported by the driver Structure of this module is in figure 8.4.

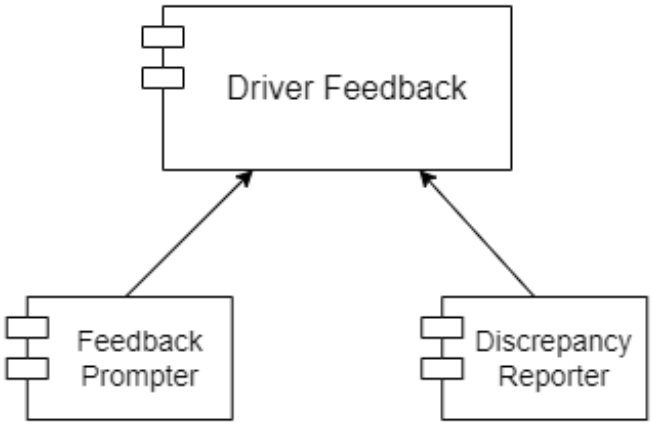


Figure 6.4: Driver Feedback - Subsystem Architecture Diagram

This figure represents the Driver Feedback Subsystem

6.4 Architectural Strategies

The architectural strategies that we will be using for the GeoSync system are given below:

6.4.1 Tools and Software

Our system will be developed using Tailwind CSS for frontend and Python with Django to implement the backend. For obtaining open source location data, we have identified Osmosis as an effective tool. Python-based tools like BeautifulSoup, Scrapy, and Selenium will be used to develop a web crawler to navigate the sites and extract location information. For geocoding, services such as Google's Geocoding API or OSM's Nominatim service may be utilised. In addition, we plan to use Google Colaboratory and Python libraries to build and train ML models for the pattern detection module. Python's versatility supports backend development, web crawling, and machine learning model building, ensuring seamless integration across various system components.

6.4.2 Client - Server Model

The project adopts a Client-Server Model where communication between the backend and frontend occurs through API requests. The backend will be developed using Python Django, whereas the frontend will utilize Tailwind CSS. This ensures effective communication and enhances overall performance and user experience.

6.4.3 Database

The chosen databases, MongoDB and PostgreSQL, are instrumental in managing location data sourced from OSM and web crawling, as well as ridesharing data, which includes essential driver feedback. The decision to employ MongoDB and PostgreSQL aligns with the project's software requirements, ensuring that the system can efficiently store, retrieve, and manipulate various datasets.

6.4.4 Interface Paradigms

The interface employed in this system will prioritize simplicity and user-friendliness, ensuring accessibility for users of all levels of expertise. Each functionality defined will be thoughtfully crafted to provide meaningful and intuitive interactions, enhancing the overall user experience.

6.4.5 Future Plans for Extension

In the future, integration of image-based approaches and the implementation of a robust data validation framework can be explored to extend the capabilities of our system. These strategic considerations are poised to enhance problem-solving methodologies and ensure a more comprehensive and reliable solution. By exploring these potential avenues, we aim to elevate the functionality and adaptability of our system.

6.5 Domain Model/Class Diagram

The system's structural design is depicted in the following class diagram, illustrating the interdependencies among key classes and their relationships within the GeoSync system. The diagram specifically highlights the involvement of two user classes (customer and driver) engaged in ride-sharing and represents their interactions. Furthermore, it shows the associations between rides and their corresponding pick-up and drop-off locations. Class Diagram is in figure 6.5.

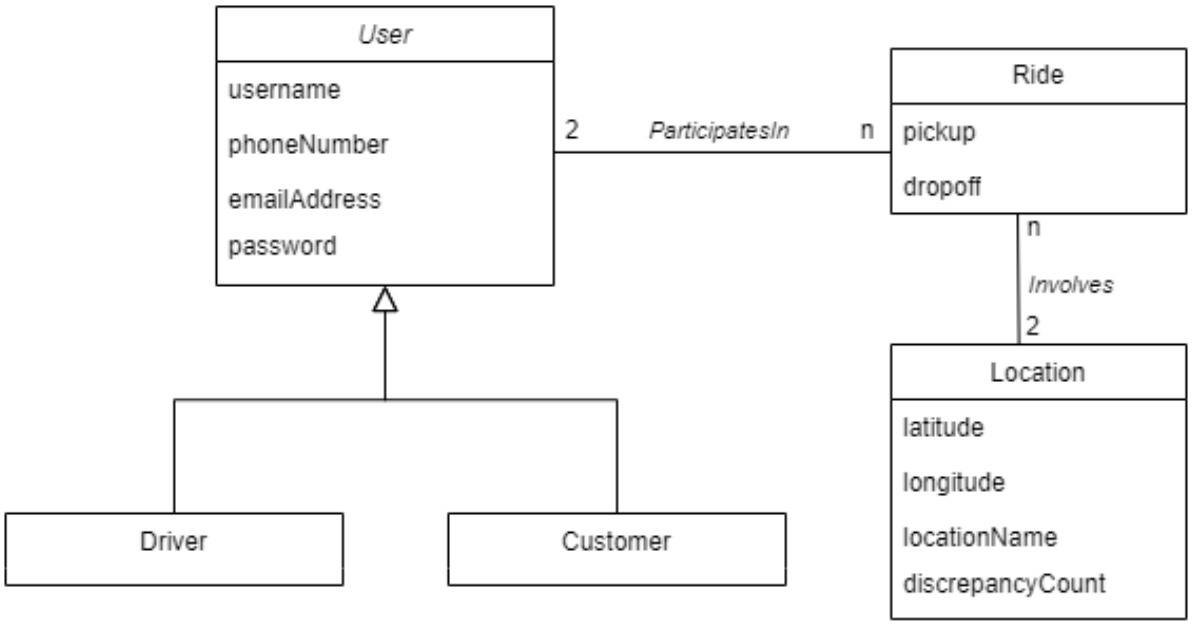


Figure 6.5: Class Diagram

This figure represents the GeoSync Class Diagram

6.6 Sequence Diagrams

In this section, the sequence diagrams for all system components are shown along with a description of their working.

6.6.1 Geosync System

This diagram represents the overall working of the system, and shows the sequence of actions triggered when a ride is booked by a user.

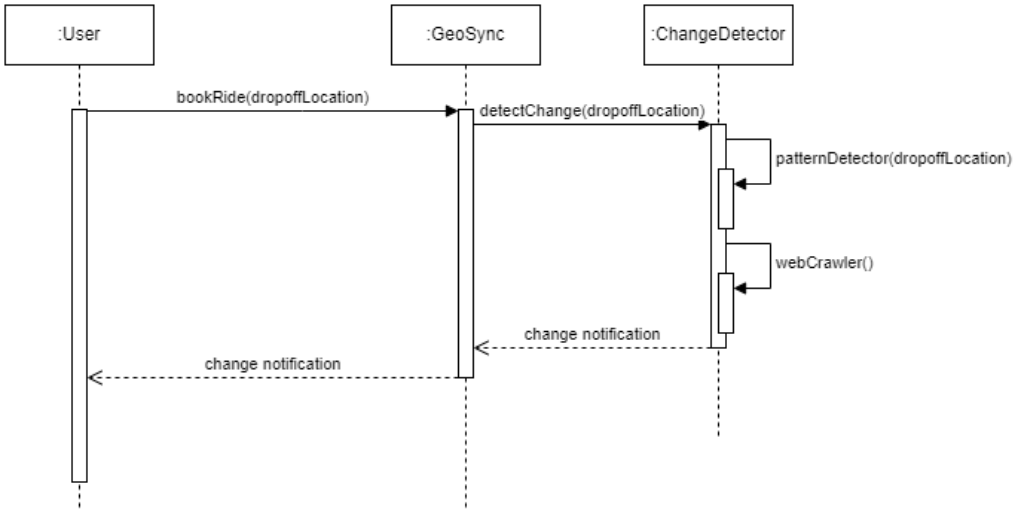


Figure 6.6: GeoSync Design Sequence Diagram

This figure represents the GeoSync System Design Sequence Diagram

6.6.2 Pattern Detector

The following diagram represents the working of the pattern detection module, initiated by a user starting a ride and entering a drop-off location.

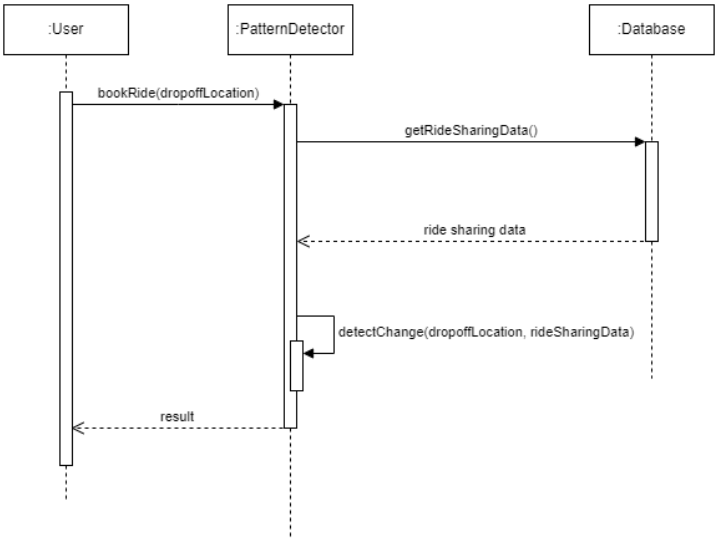


Figure 6.7: Pattern Detector Design Sequence Diagram

This figure represents the Pattern Detector Design Sequence Diagram

6.6.3 Web Crawler

The diagram illustrates the web crawler module's function. Upon user-initiated ride and drop-off location input, it extracts location information. If obtained within 10 seconds, changes are detected and returned to the user otherwise it times out, and retrieves information of last run from the database.

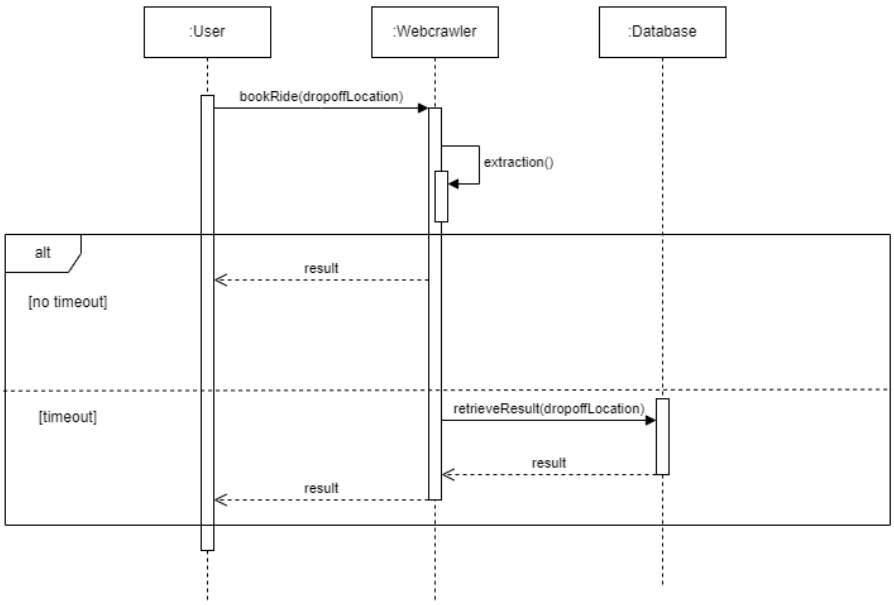


Figure 6.8: Web Crawler Design Sequence Diagram

This figure represents the Web Crawler Design Sequence Diagram

6.6.4 Driver Feedback Mechanism

The working of the driver feedback module is represented in the diagram below, showing two alternative paths, one where a discrepancy is reported by a driver and another where no discrepancy is reported.

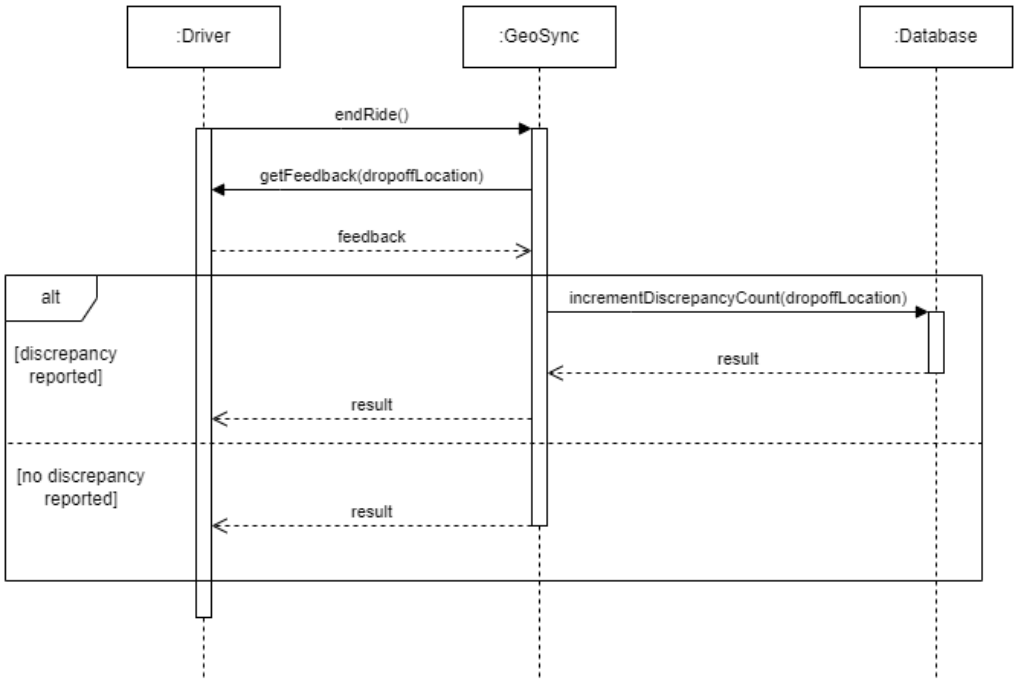


Figure 6.9: Driver Feedback Design Sequence Diagram

This figure represents the Driver Feedback Mechanism Design Sequence Diagram

6.7 Policies and Tactics

In the following section, we will delve into specific design policies and tactics employed in the development of the GeoSync system. These decisions are carefully crafted to impact the finer details of the system, focusing on the interface and implementation aspects. Unlike architectural decisions that shape the overall organisation of the system, these policies and tactics influence the intricacies of module implementation. Key considerations include discussed here include:

1. Algorithm Selection for Pattern Detection
2. Testing Strategies
3. Coding Guidelines
4. Django as the Framework Choice
5. Plans for Ensuring Requirements Traceability

6.7.1 Algorithm Selection for Pattern Detection

Siamese Graph Neural Network (GNN) is the chosen technique for pattern detection. This decision was driven by the superior performance of Siamese GNN architecture paired with Sort-k pooling technique, and the sliding window approach in capturing structural differences between graph snapshots compared to the alternatives considered. The alternatives considered include statistical and numerical pattern detection algorithms.

6.7.2 Testing Strategy

The testing strategy for the system will involve a two fold approach.

- A combination of unit testing, integration testing, and real-world scenario testing will be employed for the testing of the web app.
- Validation and Testing of the machine learning modules used will be done across various metrics like F1 score, precision accuracy and recall.

This approach will ensure the reliability, credibility and accuracy of the system as a whole.

6.7.3 Coding Guidelines

Coding guidelines and conventions provided by [24] will be followed for the development of the system. Adherence to these guidelines will streamline the collaboration among developers and facilitate maintenance of the system in future.

6.7.4 Django as the Framework Choice

The choice of Django as the preferred framework for GeoSync system development reflects a strategic policy. Django's seamless integration with Python promotes easy development, allowing the utilisation of machine learning components effortlessly. This choice ensures a robust and scalable foundation, streamlining development processes for a structured and maintainable codebase.

6.7.5 Plans for Ensuring Requirements Traceability

GeoSync is an industrial project by Uber. During the development of the GeoSync system, requirement traceability throughout the entire project lifecycle will be ensured. This strategic approach involves establishing clear links between each system component, feature, and functionality back to the overarching project requirements. Requirements traceability is emphasised because GeoSync is an industrial project by Careem strictly implementing this strategy will seamlessly align the implemented features with the

predefined project objectives consequently ensuring that the product shall pass the acceptance testing.

6.8 Conclusion

In conclusion, this chapter has provided a comprehensive overview of the GeoSync system, delving into both its high-level and low-level designs. The interactions between the various system components were demonstrated through detailed component and architecture analysis. Collectively, these elements lay the groundwork for the development and implementation of GeoSync.

Chapter 7 Implementation and Test Cases

This chapter provides the description of our current prototype. We have initiated the implementation phase of our project by creating a Django application to demonstrate the functionality of our system. The GUI includes multiple pages including the homepage where users can book a ride and the driver feedback form. The prototype includes two of the four modules of our proposed system- the acquisition of open source data and driver feedback mechanism. In addition, we have implemented the first stage of the pattern detection module, which involves the creation of structured graphs from ridesharing data for subsequent use in machine learning modeling.

7.1 Implementation

This section details the implementation process of our current prototype, including data acquisition, implementation of the driver feedback mechanism and the first stage of the pattern detection module.

7.1.1 Dataset Aquisition

For change detection, we required two main types of data- location information and ride-sharing data.

7.1.1.1 Location Data Acquisition

For our project, we used OSM, a collaborative project to create a free editable map of the world, to obtain the required location data.

7.1.1.1.1 Data Collection

The first step in the process was to acquire the necessary data. The Geofabrik's download server [25], which provides extracts of the OSM data for different regions of the world, was used to download the entire OSM dataset for Pakistan.

Once the OSM data for Pakistan was downloaded, we used Osmosis to extract the data for the specific city of interest. Osmosis is a command-line Java application for processing OSM data. It allows us to extract smaller areas from a larger dataset using a bounding box, which is a rectangular region defined by latitude and longitude coordinates. We used Osmosis to extract the data for relevant cities from the larger Pakistan dataset.

7.1.1.1.2 Data Processing

Once we had obtained the OSM data along with the required libraries, Python was employed to read and process the data. For reading OSM data, the osmium library was utilized, known for its speed and flexibility in handling OSM data through a SimpleHandler class that allows customization. We implemented a custom handler class, OSMHandler, inheriting from osm.SimpleHandler. Within this class, the node method was overridden to process nodes within the .osm file, and the tag inventory method was employed to extract data from each node, appending it to the osm data list. Subsequently, the list was converted into a pandas DataFrame, facilitating more convenient data manipulation.
This methodology allowed us to acquire a comprehensive dataset of all the named locations in the cities of interest, each with its name and coordinates.

7.1.1.2 Ride Sharing Data Creation

A comprehensive dataset was developed according to the schema provided to us by Careem. The data includes details of each ride, including userID, pick-up and drop-off coordinates and location name. Furthermore, we aimed at developing a detailed dataset that matches the diversity of ride-sharing in the real world.
To achieve this, we considered an extensive range of use cases to ensure the inclusion of all possible scenarios. These scenarios encompass both frequent and infrequent users of the app to capture the full spectrum of user engagement. We also took into account the variability in user behavior and patterns, acknowledging that each user interacts with the app in a unique way. These patterns, particularly deviations from the norm, serve as key indicators for our pattern detection algorithm. This enables the algorithm to identify changes in the location of specific POIs.

7.1.2 Driver Feedback

This subsection describes the implementation of driver feedback module which acts as a feedback loop for detecting discrepancies in the location data based on customer rides.

7.1.2.1 Feedback Form

Whenever a ride ends, the driver will be directed to a form asking if the location which the customer chose was accurate in the system's location data. The form is in Figure 8.4. In the database storing location information, a column for discrepancy count has been kept for each location. The discrepancy count for that location will be incremented in the database if a change is reported.

7.1.2.2 Detecting Change in Location

If multiple drivers report discrepancies between the customer's chosen location and a certain POI, that is an indication of an inaccurate location in the system's location data. As discrepancy count of each location is being recorded upon every driver feedback, once the discrepancy count reaches 5, we signify that a change in location has been detected.

7.1.3 Pattern Detection

The following subsection entails our pattern detection module which includes data preparation, graph creation, model training and evaluation.

7.1.3.1 Queried Relevant Data

Upon running this module, rows with a specific drop-off location name, such as "FAST NUCES Lahore Campus," are filtered from the ride-sharing data for analysis. Subsequently, these filtered rows are referred to as the "queried data."

7.1.3.2 Forming Temporal Sequences

The previously identified queried data is then segmented into temporal sequences, each within a predetermined time window, whenever this module is executed. This time window will be subject to periodic evolution based on model performance. After segmentation, each sequence is transformed into directed graphs in the subsequent step.

7.1.3.3 Directed Graphs

Within each temporal sequence, the data will encapsulate rides to a specific POI during a designated time window. The POI in this data chunk is treated as a node within the directed graph, accompanied by all unique users present in the corresponding data sequence. Utilizing the networkx Python library, directed graph structures are initialized, creating one for each distinct time window. Each ride within the sequence is translated into an edge connecting the user to the POI, with the timestamp serving as the edge weight.

This process facilitates the transformation of ride-sharing data into structured graphs, laying the foundation for subsequent analysis and machine learning modeling steps.

7.1.3.4 Model Training

In this phase, the model is trained using pairs of consecutive directed graphs, each representing the ride-sharing activity within different but consecutive time windows. This training process aims to optimize the network's parameters to identify and understand the evolution of mobility patterns across these time windows.

To facilitate this, a dataset comprising pairs of graph representations is utilized. Each pair consists of two graphs: one representing ride-sharing activity at time (t) and the other at time (t+1). The objective during training is to minimize the difference between the model's output on these graph pairs and the target similarity measure, which in this context could be a representation of the change or stability of mobility patterns between the consecutive time windows.

The loss function employed, Mean Squared Error (MSE), quantifies the deviation of the model's similarity predictions from the target values. This loss is backpropagated through the network to update the model parameters using the Adam optimizer, a widely used optimization algorithm that adjusts the weights to minimize the loss function.

Training involves multiple epochs, where an epoch is one complete pass through the entire training dataset. After each epoch, the average loss across all graph pairs is computed and monitored to track the learning progress.

7.1.3.5 Evaluation and Model Performance

Following training, the model's performance is evaluated on a separate set of graph pairs that were not seen during the training phase. This evaluation step is crucial for assessing the model's generalization ability - its performance on new, unseen data.

The evaluation process is similar to the training process but without the optimization step (i.e., no back-propagation or weight updates). The model predicts the similarity between graph pairs in the test set, and the loss function is used to compute the error between these predictions and the target values. This error metric provides insight into the model's accuracy and reliability in detecting changes in mobility patterns.

7.1.4 Web Crawler

The initial phase employs Selenium WebDriver, complemented by the BeautifulSoup library, to automate the navigation of specified URLs on the website [26] targeting the Gulberg area. This combination is instrumental in parsing HTML content to extract street-level addresses. Selenium initiates ChromeDriver, facilitating browser interaction to scrape data, while BeautifulSoup parses the loaded content, pinpointing addresses within specific div tags for collection.

7.1.4.1 Automated Data Extraction

Upon the successful collection of addresses, the submodule advances to the geocoding phase, utilizing the Google Maps API to convert the scraped addresses into precise latitude and longitude coordinates. This transformation is important for maintaining our system's location database with accurate, real-world geographical information. Through initializing a Google Maps API client with an API key, the module processes each address, extracting and recording the geographical coordinates upon successful geocoding. Oh

7.1.4.2 Geocoding for Accurate Coordinates

Upon the successful collection of addresses, the submodule advances to the geocoding phase, utilizing the Google Maps API to convert the scraped addresses into precise latitude and longitude coordinates. This transformation is important for maintaining our system's location database with accurate, real-world geographical information. Through initializing a Google Maps API client with an API key, the module processes each address, extracting and recording the geographical coordinates upon successful geocoding.

7.2 Test Case Design and Description

This section outlines the common input constraints applicable to all associated test cases for our system GeoSync. It also includes any shared environmental requirements, special procedural needs along with any dependencies among cases.

7.2.1 Rider Login Test Case

Rider Login			
Test Case ID:	*1*	**QA Test Engineer:**	*Hadeeqa Imran*
Test Case Version:	*1*	**Reviewed By:**	*Zainab Rashid*
Test Date:	*April 20, 2024*	**Use Case Reference(s):**	*Login*
Revision History:	*None*		
Objective:	*To verify if a rider is able to successfully login in the system.*		
Product/Ver/Module:	*Login Module*		
Environment:	*Operating System: Ubuntu 22.04.1 LTS*		
Assumptions:	*The internet connection is stable and the website is accessible.* *The system's backend, including the database containing user credentials, is operational.*		

Continued on next page

Rider Login		
Pre-Requisite:	*None*	
Step No.	**Execution description**	**Procedure result**
1	*Rider clicks on the rider login button.*	*System displays the rider login page.*
2	*Rider enters their credentials and clicks the login button.*	*System logs in the rider and loads the rider home page.*
Comments:	The test case is passed. Our system is working according to the requirements specified.	
Passed		

7.2.2 Driver Login Test Case

Driver Login			
Test Case ID:	*2*	**QA Test Engineer:**	*Huzaifa Faheem*
Test Case Version:	*1*	**Reviewed By:**	*Zainab Rashid*
Test Date:	*April 20, 2024*	**Use Case Reference(s):**	*Login*
Revision History:	*None*		
Objective:	*To verify if a driver is able to successfully login in the system.*		
Product/Ver/Module:	*Login Module*		
Environment:	*Operating System: Ubuntu 22.04.1 LTS*		
Assumptions:	*The internet connection is stable and the website is accessible. The system's backend, including the database containing user credentials, is operational.*		
Pre-Requisite:	*None*		
Step No.	**Execution description**	**Procedure result**	
1	*Driver clicks on the driver login button.*	*System displays the driver login page*	
2	*Driver enters their credentials and clicks the login button.*	*System logs in the driver and loads the driver home page.*	
Comments:	The test case is passed. Our system is working according to the requirements specified.		
Passed			

7.2.3 Rider Signup Test Case

Rdier Sign Up			
Test Case ID:	*3*	**QA Test Engineer:**	*Zainab Rashid*
Test Case Version:	*1*	**Reviewed By:**	*Hadeeqa Imran*
Continued on next page			

Rider Sign Up			
Test Date:	*April 20, 2024*	**Use Case Reference(s):**	*Sign Up*
Revision History:	*None*		
Objective:	*To verify if a rider is able to successfully register to the system.*		
Product/Ver/Module:	*Sign Up Module*		
Environment:	*Operating System: Ubuntu 22.04.1 LTS*		
Assumptions:	*The internet connection is stable and the website is accessible. The system's backend, including the database containing user credentials, is operational.*		
Pre-Requisite:	*None*		
Step No.	**Execution description**	**Procedure result**	
1	*Rider clicks on the rider register button.*	*System displays the rider register page.*	
2	*Rider enters their credentials and clicks the register button.*	*System registers and logs in the rider and loads the rider home page.*	
Comments:	The test case is passed. Our system is working according to the requirements specified.		
Passed			

7.2.4 Driver Signup Test Case

Driver Sign Up			
Test Case ID:	*4*	**QA Test Engineer:**	*Zainab Rashid*
Test Case Version:	*1*	**Reviewed By:**	*Huzaifa Faheem*
Test Date:	*April 20, 2024*	**Use Case Reference(s):**	*Book Ride Module*
Revision History:	*None*		
Objective:	*To verify if a rider is successfully able to book a ride.*		
Product/Ver/Module:	*Sign Up Module*		
Environment:	*Operating System: Ubuntu 22.04.1 LTS*		
Assumptions:	*The internet connection is stable and the website is accessible. The system's backend, including the database containing user credentials, is operational.*		
Pre-Requisite:	*None*		
Step No.	**Execution description**	**Procedure result**	
1	*Driver clicks on the driver register button.*	*System displays the driver register page.*	

Continued on next page

Driver Sign Up		
2	*Driver enters their credentials and clicks the register button.*	*System registers and logs in the driver and loads the driver home page.*
Comments:	The test case is passed. Our system is working according to the requirements specified.	
Passed		

7.2.5 Book Ride Test Case

Book Ride			
Test Case ID:	*5*	**QA Test Engineer:**	*Huzaifa Faheem*
Test Case Version:	*1*	**Reviewed By:**	*Hadeeqa Imran*
Test Date:	*April 20, 2024*	**Use Case Reference(s):**	*Book Ride*
Revision History:	*None*		
Objective:	*To verify if a rider is successfully able to book a ride.*		
Product/Ver/Module:	*Book Ride Module*		
Environment:	*Operating System: Ubuntu 22.04.1 LTS*		
Assumptions:	*The internet connection is stable and the website is accessible. The system's backend, including the database containing user credentials, is operational.*		
Pre-Requisite:	*None*		
Step No.	**Execution description**	**Procedure result**	
1	*Rider selects a pick up location from pick up location drop down.*	*Pick up location is selected.*	
2	*Rider selects a drop off location from drop off location drop down.*	*Drop off location is selected.*	
3	*Rider clicks the start ride button.*	*System runs the change detector module against the selected drop off location.*	
Comments:	The test case is passed. Our system is working according to the requirements specified.		
Passed			

7.2.6 Notify Rider Test Case

Notify Rider			
Test Case ID:	*6*	**QA Test Engineer:**	*Zainab Rashid*
Test Case Version:	*1*	**Reviewed By:**	*Huzaifa Faheem*
Test Date:	*April 20, 2024*	**Use Case Reference(s):**	*Notify User*
Revision History:	*None*		
Objective:	*To verify that the system accurately notifies the rider of location changes.*		
Product/Ver/Module:	*Notification Module*		
Environment:	*Operating System: Ubuntu 22.04.1 LTS*		
Assumptions:	*The internet connection is stable and the website is accessible. The system's backend, including the database containing user credentials, is operational.*		
Pre-Requisite:	*Rider has booked a ride and the change detector module has run against the drop off location entered.*		
Step No.	**Execution description**	**Procedure result**	
1	*Rider clicks the notification page button.*	*System retrieves the result of the change detector module and displays a notification that location has changed if a location change is detected.*	
Comments:	The test case is passed. Our system is working according to the requirements specified.		
Passed			

7.2.7 Driver Feedback Test Case

Driver Feedback			
Test Case ID:	*7*	**QA Test Engineer:**	*Huzaifa Faheem*
Test Case Version:	*1*	**Reviewed By:**	*Hadeeqa Imran*
Test Date:	*April 20, 2024*	**Use Case Reference(s):**	*Provide Feedback*
Revision History:	*None*		
Objective:	*To verify that the system correctly gets and stores the driver feedback after the end of each ride.*		
Product/Ver/Module:	*Driver Feedback Module*		
Environment:	*Operating System: Ubuntu 22.04.1 LTS*		
Assumptions:	*The internet connection is stable and the website is accessible. The system's backend, including the database containing user credentials, is operational.*		

Continued on next page

Driver Feedback		
Pre-Requisite:	*A ride has been completed and the driver is shown the driver feedback page.*	
Step No.	**Execution description**	**Procedure result**
1	*The driver chooses if they saw a discrepancy in the drop off location of their last ride.*	*Systems increments the discrepancy count for a specific drop off location if the driver chose yes.*
Comments:	The test case is passed. Our system is working according to the requirements specified.	
Passed		

7.2.8 Pattern Detector Test Case

Pattern Detector			
Test Case ID:	*8*	**QA Test Engineer:**	*Hadeeqa Imran*
Test Case Version:	*1*	**Reviewed By:**	*Zainab Rshid*
Test Date:	*April 20, 2024*	**Use Case Reference(s):**	*Detect Location Change*
Revision History:	*None*		
Objective:	*To verify that the system correctly gets and stores the driver feedback after the end of each ride.*		
Product/Ver/Module:	*Pattern Detector Module*		
Environment:	*Operating System: Ubuntu 22.04.1 LTS*		
Assumptions:	*The internet connection is stable and the website is accessible. The system's backend, including the database containing user credentials, is operational.*		
Pre-Requisite:	*Rider has entered the pick up and drop off locations.*		
Step No.	**Execution description**	**Procedure result**	
1	*Rider clicks the start ride button.*	*The system triggers the change detector module for a specified location, which subsequently activates the pattern detector module. The pattern detector module accurately identifies significant changes in patterns for the specified drop off location, generates an appropriate change score, and returns this score to the change detector module.*	
Comments:	The test case is passed. Our system is working according to the requirements specified.		

Continued on next page

Pattern Detector
Passed

7.2.9 Web Crawler Test Case

Web Crawler			
Test Case ID:	*9*	**QA Test Engineer:**	*Huzaifa Faheem*
Test Case Version:	*1*	**Reviewed By:**	*Hadeeqa Imran*
Test Date:	*April 20, 2024*	**Use Case Reference(s):**	*Detect Location Change*
Revision History:	*None*		
Objective:	*To verify that the web crawler module is triggered automatically once a ride is booked.*		
Product/Ver/Module:	*Web Crawler Module*		
Environment:	*Operating System: Ubuntu 22.04.1 LTS*		
Assumptions:	*The internet connection is stable and the website is accessible. The system's backend, including the database containing user credentials, is operational.*		
Pre-Requisite:	*Rider has entered the pick up and drop off locations.*		
Step No.	**Execution description**	**Procedure result**	
1	*Rider clicks the start ride button.*	*The system automatically initiates the change detector module for a specified location, which in turn activates the web crawler module. The web crawler module effectively scrapes location data for the specified drop-off location from a designated website, compares it with the data in the database, and returns the comparison result to the change detector module.*	
Comments:	The test case is passed. Our system is working according to the requirements specified.		
Passed			

7.2.10 Change Detector Test Case

Change Detector			
Test Case ID:	*10*	**QA Test Engineer:**	*Hadeeqa Imran*
Test Case Version:	*1*	**Reviewed By:**	*Zainab Rashid*
Test Date:	*April 20, 2024*	**Use Case Reference(s):**	*Detect Location Change*
Revision History:	*None*		
Objective:	*To verify that the change detector module functions correctly by triggering the pattern detector and web crawler modules, retrieving results from each, and generating a unified output.*		
Product/Ver/Module:	*Change Detector Module*		
Environment:	*Operating System: Ubuntu 22.04.1 LTS*		
Assumptions:	*The internet connection is stable and the website is accessible. The system's backend, including the database containing user credentials, is operational.*		
Pre-Requisite:	*Rider clicks the start ride button.*		
Step No.	**Execution description**	**Procedure result**	
1	*Rider clicks the start ride button.*	*Upon initiating the ride, the system automatically activates the change detector module. This module triggers the pattern detector and web crawler modules, gathers their outputs, and incorporates the driver discrepancy count. It then calculates a composite score to determine if the location has changed.*	
Comments:	The test case is passed. Our system is working according to the requirements specified.		
Passed			

7.2.11 Test Cases Summary Table

The metrics indicate a successful testing outcome with all 10 test cases executed and passed, resulting in zero defects. The Test Case Defect Density is 0%, showcasing no issues found, and Test Case Effectiveness is 100%, reflecting the tests' accuracy and thoroughness

Metric	**Purpose**
Number of Test Cases	10
Number of Test Cases Passed	10
Number of Test Cases Failed	0
Test Case Defect Density	0%
Test Case Effectiveness	100%
Traceability Matrix	-

7.3 Conclusion

To conclude, the seventh chapter details the transition from theory to a fully operational system. It covers the system's implementation, including dataset acquisition, the driver feedback mechanism, pattern detection, and web crawler modules. Additionally, the chapter introduces test case design and execution, which assess module performance across accuracy and integration, ensuring system robustness and cohesive functionality.

Chapter 8 User Manual

In this chapter, a complete user manual for our system GeoSync is presented.

8.1 Introduction of Geosync

Our website demonstrates a hybrid change detection system which is essential for maintaining up-to-date location data. This system is crucial for services like Careem that depend on accurate Points of Interest (POIs).

8.1.1 Overview of Change Detector System

The Change Detector System is a hybrid solution that tackles the problem of outdated POIs, which may change due to rebranding, relocations, or closures. By automatically detecting these changes, the system ensures that the location data provided to users is current and reliable.

8.1.2 Purpose and Benefits of Change Detector System

The purpose of the Change Detector System is to enhance the accuracy of location-based services provided to Careem's drivers and riders. This leads to an improved user experience by minimizing navigation errors and enhancing efficiency in route planning. The benefits of this system include avoiding potential lost business due to inaccurate data.

8.2 User Interaction

This section details how users interact with the GeoSync system to make the most of the up-to-date data provided.

8.2.1 Registration and Account Creation

Both drivers and riders use a similar process to register their accounts.

8.2.1.1 Choosing Your Role

Upon visiting the GeoSync website, users are presented with two distinct options on the registration page: Driver or Rider. This initial choice tailors the registration process to suit the specific needs and services relevant to each role.

8.2.1.2 Instructions

- **Access the GeoSync website:** Navigate to the homepage.
- **Choose your role:** Click on ”Driver” or ”Rider” based on your intended use of the platform. Each option will lead you to a tailored registration form.

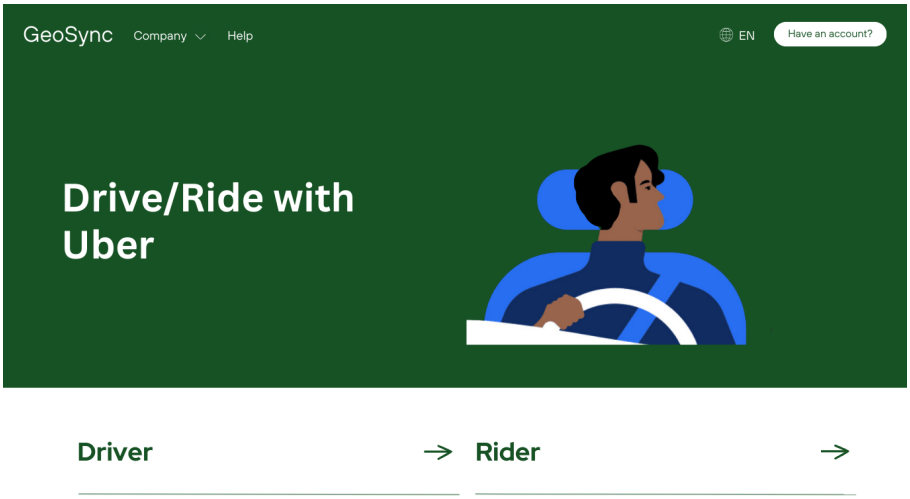


Figure 8.1: Signup Form

This figure represents the Signup form of the system

- **Driver and Rider Account Creation:** After selecting the ”Driver” or ”Rider” role, users are prompted to enter their details on the registration form. This form typically requires personal information such as name, email address, and a chosen password.

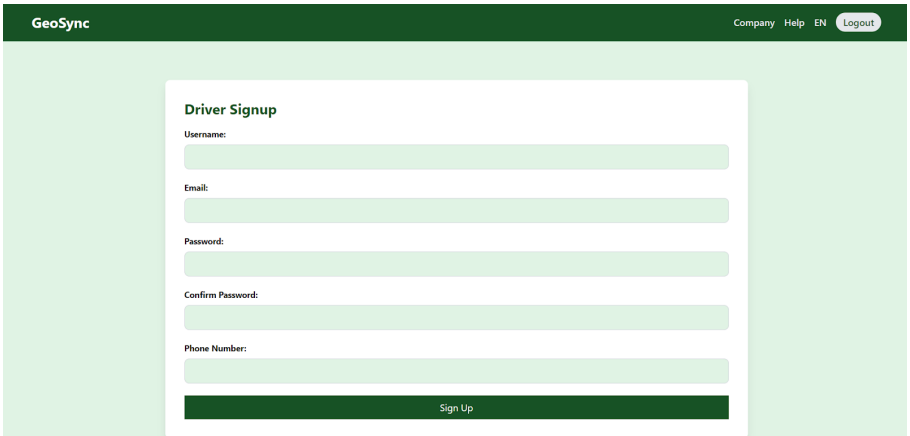


Figure 8.2: Signup Page

This figure represents the Signup page of the system

8.2.2 Login for Drivers and Riders

Both drivers and riders use a similar process to log into their accounts.

8.2.2.1 Instructions

- **Visit the Login Page:** Navigate via the link provided or by selecting the 'Login' option on the homepage.
- **Enter Username and Password:** Fill in your credentials in the respective fields.
- **Access the Account:** Click on the "Login" button to proceed to your dashboard where you can engage with the features of the system.

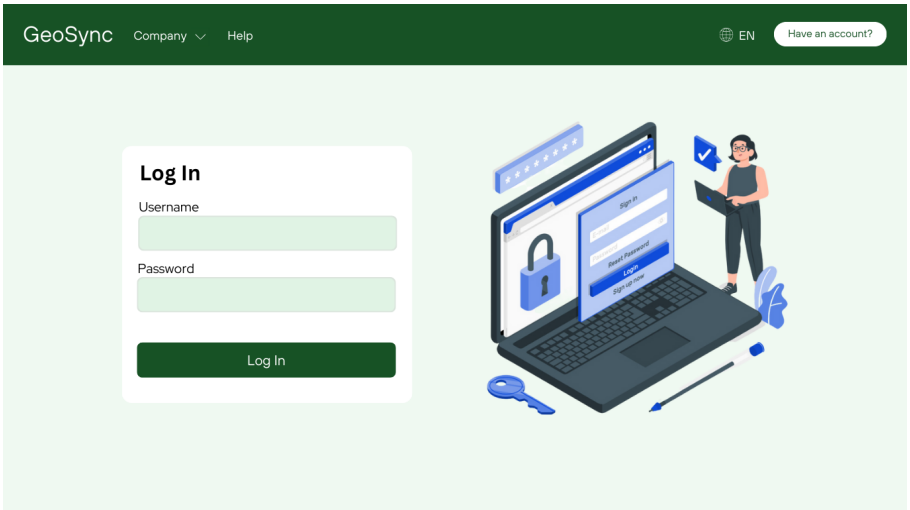


Figure 8.3: Login Page

This figure represents the login page of the system

8.2.3 Navigating the Website

This section details how to navigate the GeoSync website.

8.2.3.1 Driver Homepage Functionality

Once logged in, drivers are directed to their homepage, which includes functionalities aimed at enhancing their interaction with the GeoSync system.

8.2.3.2 Instructions

- **Report Location Discrepancy:** Navigate to the "Report Location Discrepancy" section on the homepage.
- **Select Location:** Use the drop-down menu to choose the location you wish to report. Indicate Discrepancy: Select 'Yes' or 'No'.
- **Submit Report:** Click on the "Submit Report" button to send your feedback. 8.4

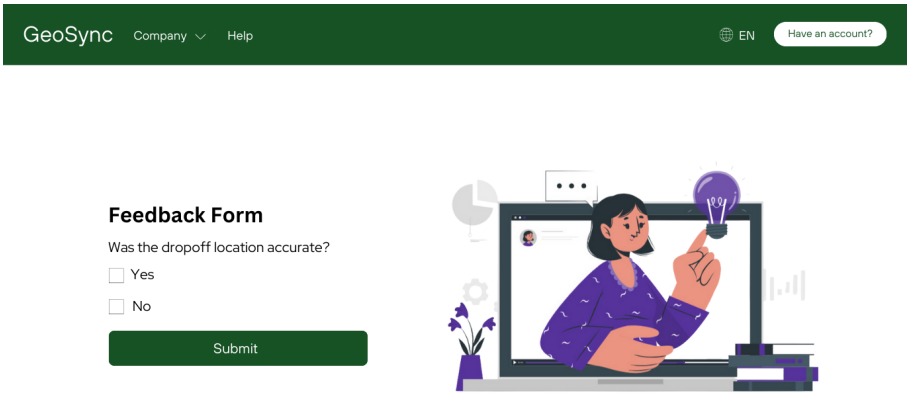


Figure 8.4: Driver Homepage

This figure represents the driver feedback of the system

8.2.3.3 Rider Homepage Functionality

Riders access a simple interface for booking rides, which incorporates the change detection module.

8.2.3.4 Instructions

- **Enter Pick-Up Location:** Select your current location from the drop-down menu.
- **Enter Drop-Off Location:** Choose your desired destination from the drop-down menu.
- **Start the Ride:** Click the ”Start Ride” button.
- **Change Detection Notification:** If any changes are detected at the drop-off location: You will receive a notification on a subsequent page informing you of any changes detected in the location data. 8.5

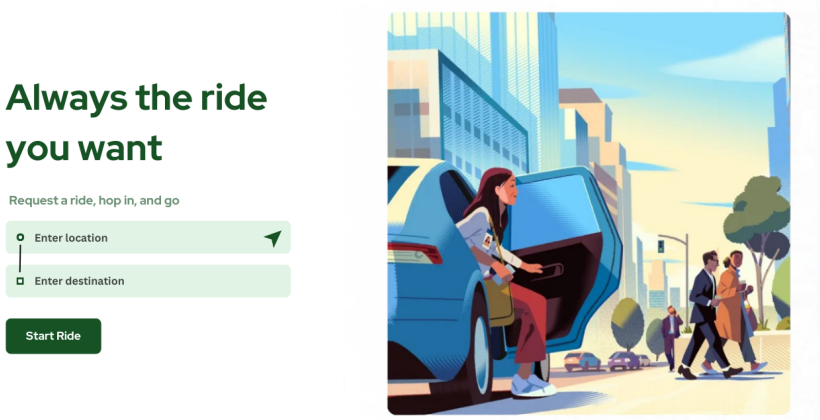


Figure 8.5: Rider Homepage

This figure represents the the rider homepage in GeoSync

Chapter 9 Conclusion

The dynamic nature of locations poses significant challenges for ride-sharing services like Careem, ranging from customer dissatisfaction to operational inefficiencies. Our project, GeoSync, aims to tackle this challenge by developing an automated Points of Interest (POI) change detection system.

Through a comprehensive literature review, we developed a multi-faceted solution that utilizes web crawlers, feedback from Careem Captains, and open-source maps. Our proposed system acquires open-source location data and comprises three distinct modules for POI change detection: pattern recognition, web scraping, and a driver feedback system. The knowledge gained from the literature review and brainstorming sessions informed the creation of use cases for the change detection modules. Additionally, we created initial versions of the GUI for the Django app, which will integrate all the change detection modules. Subsequently, we developed the proposed methodology and used it to create system architecture diagrams. Lastly, we began work on the prototype, which represents our progress to date.

The project has made progress in various key areas. The initial focus was on the acquisition of open-source data, specifically utilizing the OSM project to obtain essential location information. It was done successfully by using osmosis and python. Incorporating user feedback into the system was important, so the driver feedback module was implemented, allowing drivers to report discrepancies in customer-chosen locations. In parallel, dataset preparation for the pattern detection module was done. This involves the creation of a comprehensive dataset reflecting diverse ride-sharing scenarios and subsequent graph creation to represent spatial-temporal patterns. These integral components collectively form the progress with regards to implementation of the project during FYP-1. For our current prototype, a Django application has been created to demonstrate the functionality of the system.

A major challenge encountered during development was the delay from Careem's end in the provision of ride-sharing data. To save the development process from facing setbacks, we obtained the schema and leveraged python to generate a comprehensive dataset ourselves. This allowed us to maintain momentum in the workflow, enabling continued progress and on time development.

To conclude, the project has successfully addressed the challenges of detecting changes in location data in ride-sharing services. All modules, including data acquisition, driver feedback, web crawling, and pattern detection, have been integrated and tested, ensuring that GeoSync is ready for deployment. This will enhance the efficiency and user experience for Careem by streamlining its operations and providing reliable location-based services.

Bibliography

[1] M. Kumar, R. Bhatia, and D. Rattan, "A survey of web crawlers for information retrieval," *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, vol. 7, no. 6, p. e1218, 2017.

[2] D. Ahlers and S. Boll, "Adaptive geospatially focused crawling," in *Proceedings of the 18th ACM conference on Information and knowledge management*, pp. 445–454, 2009.

[3] D. Ahlers and S. Boll, "Retrieving address-based locations from the web," in *Proceedings of the 5th Workshop on Geographic Information Retrieval*, pp. 27–34, 2008.

[4] D. Ahlers and S. Boll, "Location-based web search," *The Geospatial Web: How Geobrowsers, Social Software and the Web 2.0 are Shaping the Network Society*, pp. 55–66, 2007.

[5] OpenStreetMap, "About openstreetmap," 2023. OpenStreetMap, 2023. [Online]. Available: https://www.openstreetmap.org/about [Accessed 2023-10-08].

[6] M. Haklay, "How good is volunteered geographical information? a comparative study of openstreetmap and ordnance survey datasets," *Environment and planning B: Planning and design*, vol. 37, no. 4, pp. 682–703, 2010.

[7] R. T. Olszewski, *Generalized feature extraction for structural pattern recognition in time-series data*. Carnegie Mellon University, 2001.

[8] M. Corneli, P. Latouche, and F. Rossi, "Multiple change points detection and clustering in dynamic networks," *Statistics and Computing*, vol. 28, pp. 989–1007, 2018.

[9] D. Sulem, H. Kenlay, M. Cucuringu, and X. Dong, "Graph similarity learning for change-point detection in dynamic networks," *arXiv preprint arXiv:2203.15470*, 2022.

[10] J. Revaud, M. Heo, R. S. Rezende, C. You, and S.-G. Jeong, "Did it change? learning to detect point-of-interest changes for proactive map updates," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 4086–4095, 2019.

[11] I. de Gélis, S. Lefèvre, and T. Corpetti, "Dc3dcd: unsupervised learning for multiclass 3d point cloud change detection," *arXiv preprint arXiv:2305.05421*, 2023.

[12] L. W. Yeow, R. Low, Y. X. Tan, and L. Cheah, "Point-of-interest (poi) data validation methods: an urban case study," *ISPRS International Journal of Geo-Information*, vol. 10, no. 11, p. 735, 2021.

[13] OneMap, "Onemap," 2023. Singapore Land Authority, 2023. [Online]. Available: https://www.sla.gov.sg/geospatial-development-and-services/onemap. [Accessed: 2023-09-30].

[14] G. Touya, V. Antoniou, A.-M. Olteanu-Raimond, and M.-D. Van Damme, "Assessing crowd-sourced poi quality: Combining methods based on reference data, history, and spatial relations," *ISPRS International Journal of Geo-Information*, vol. 6, no. 3, p. 80, 2017.

[15] S. Meek, M. J. Jackson, and D. G. Leibovici, "A flexible framework for assessing the quality of crowdsourced data," 2014.

[16] Lyft, "About lyft." Lytf. [Online]. Available: https://help.lyft.com/hc/ru/all/categories/3009525099-about-lyft. [Accessed: 2023-10-06].

[17] A. Yuen, "How lyft creates hyper-accurate maps from open-source maps and real-time data." Lyft Engineering, 2019. [Online]. Available: https://shorturl.at/ipyU1. [Accessed: 2023-10-06].

[18] Strava, "Record. sweat. share. kudos.." Strava. [Online]. Available: https://www.strava.com/about. [Accessed: 2023-10-06].

[19] S. Support, "About strava maps." Strava Support, 2022. [Online]. Available: https://support.strava.com/hc/en-us/articles/216917717-About-Strava-Maps. [Accessed: 2023-10-06].

[20] Grab, "Charging forward together." Grab. [Online]. Available: https://www.grab.com/sg/about/. [Accessed: 2023-10-06].

[21] GrabCom, "Grab." GrabCom, 2023. [Online]. Available: https://wiki.openstreetmap.org/wiki/Grab. [Accessed: 2023-10-06].

[22] Rosie, "Segment matching issues." Strava Support, 2023. [Online]. Available: https://support.strava.com/hc/en-us/articles/216918187-Segment-Matching-Issues. [Accessed: 2023-10-06].

[23] J. Russell, "Grab maps openstreetmap thailand and southeast asia."

TechCrunch, 2018. [Online]. Available: https://techcrunch.com/2018/12/19/grab-maps-osm-thailand-southeast-asia/. [Accessed: 2023-10-06].

[24] G. v. Rossum, B. Warsaw, and A. Coghlan, "Python enhancement proposals," Jul 2001. [Online]. Available: https://peps.python.org/pep-0008. [Accessed: 2023-10-06].

[25] GeoFabrik, "Download openstreetmap data for this region: Pakistan," 2018. [Online]. Available: https://download.geofabrik.de/asia/pakistan.html. [Accessed: 2023-12-10].

[26] "Lahore commercial companies index," Jun 2022. [Online]. Available: https://lcci.pk/. [Accessed: 2024-03-03].